Rotation synthesis: magic compression via global SNAP objective

Friends
(Dated: October 22, 2026)

In fault-tolerant quantum computation, one often needs to synthesize arbitrary single-qubit rotations from a discrete gate set such as Clifford+T. It is commonly stated in the literature that these rotations are costly, especially when they need to be approximated to high precision. We introduce a compilation method that can substantially reduce the cost of rotations for many applications of interest, such as state preparation and Trotterized time evolution. Instead of using the common approach in which the rotation angles are supplied from a higher-level algorithm and then fed into a gate synthesis method, we directly synthesize the subroutine using the Clifford+T gate set. Across different applications (such as state preparation, time evolution, and quantum phase estimation) and models, we consistently observe at least an order of magnitude reduction in the number of Cliffords and T-gates. For some applications such as state preparation, we observe the average T-gate counts per rotation approaching 1 or lower.

I. INTRODUCTION

general comments: lets remove all discussion of taper/QPE from the paper. Our results will exclusively be infinite results, but we can mention that these estimators are even more natural in finite setting. I am going to be moving things to appendices, feel free to adjust later; my goal right now is to present snap overview in section 2, and to have it be as general/untechnical as possible. there is also some inconsistent notation I am using, we will fix this later. I also changed the title for now, although we can discuss this further. I am not sure if we are in agreement on the term "magic compression". Sohan: I gave \sg the color blue

moved rotation review to appendix. moved continuous compile and environments to appendix; these will need substantial modifications for stateprep. move hessian stateprep details to appendix. moved snap technical details to appendix. In the end, I think we can remove much from the appendix, and simply cite the relevant literature, most of this is fairly well-established

One of the important applications of quantum computer is simulation of physical systems that appear in nature [1, 2]. In recent years, there has been a considerable amount of progress in the development of quantum computing hardware, enabling experiments that are capable of probing the behavior of quantum systems that are difficult to simulate classically [3–9]. However, the noise in these systems sets a hard limit on the number of operations that can be applied before the result becomes unreliable. It is widely accepted that some form of quantum error correction is necessary to overcome this issue [10].

While quantum error correction can reduce the error rate, using it comes with its own set of challenges. For instance, the set of fault-tolerant logical gates must be necessarily discrete [11], necessitating the approximation of continuous gate set by the discrete ones. While this problem can be solved by synthesis methods that asymptotically incur a polylogarithmic overhead in the precision, the actual cost of these operations remain substantial [12–19].

A commonly discussed application of quantum computer is simulation of Trotterized dynamics [17, 18, 20–25]. This entails applying rotations whose angles depend sensitively on various parameters, such as the underlying Hamiltonian and the evolution time. Such rotations, in turn, consume a large number of non-Clifford gates (e.g., T-gates), which tend to be more costly than the Clifford gates [26–32], although recent progress on magic state preparation has narrowed this gap [33–37]. Thus there has been a considerable amount of efforts aiming to reduce the cost of these rotations [18, 38–53].

It should be noted that in almost all of these works there is an implicit assumption on how one would compile a quantum algorithm (such as time evolution) down to a set of elementary set of fault-tolerant gates. This would be done by approximating a target unitary by a sequence of simple gates, such as rotations and two-qubit gates. The continuous gates are subsequently compiled in terms of the elementary gate set. In this process, there is often a target accuracy for the unitary at hand, which translates into the target accuracy of the individual rotations.

The synthesis approaches in the literature are inherently local approaches, taking each rotation gate independently and synthesizing it to some precision ϵ [14, 15, 54]. However, for important application such as state preparation and time evolution, it is understood that pieces of a quantum circuit can classically simulated using tools such as tensor networks. Indeed, to implement state preparation on a quantum computer, one often uses classical tools to approximate the ground state, which is then compiled to rotations and two-qubit gates to some precision ϵ. Similarly, time evolution can be simulated classically for small times t. This fact has previously been used to compress evolution circuits for fixed evolution error ϵ. The essential premise of our rotation synthesis approach is to take advantage of our ability to classically simulate these pieces, with the synthesis objective going from a local objective to a global objective.

There are certain special rotation that are low-magic, and can be implemented with few T gates; ideally we would synthesize our rotations to these special ones as much as possible. We refer to such a replacement as

a SNAP. However, one can expect many SNAPs to increase the error of the synthesized circuit, which is why local approaches will rarely implement such a replacement. However, with access to the global objective, we can consider re-optimizing the rotations in the remainder of the circuit so as to minimize the global error. In this regard our snap criteria is a global objective: we implement snaps that upon re-optimization of the remaining gates gives a global error below ϵ.

The most surprising aspect of our approach is that it yields a substantial reduction in Clifford and T-count using only one additional logical qubit per rotation. For comparison with the existing approaches, see Fig. 6. In our opinion, this is unexpected because the gate synthesis methods in the literature [14, 15, 54] are essentially optimal down to the constants. Thus our work shows that there is a large room for improvement, breaking past the barrier set by these methods, by considering an optimization of the target unitary directly in terms of Clifford + T.

this paragraph should be modified At a high level, our approach can be viewed as an amalgamation of two different approaches studied in the literature, one aiming to approximate a unitary with fewer rotation gates and the other aiming to approximate continuous gates with the minimal number of Clifford and T gates. For the former, the main idea is to use tensor networks to variationally optimize the target state preparation circuit/unitary. In particular, recent works have shown that this approach can reduce the number of rotation gates substantially compared to their Trotterized counterpart [55–63]. For the latter, we note that there are brute-force enumeration methods for finding the optimal Clifford+T decomposition of unitaries [64, 65]. While these methods have a runtime scaling polynomially with the inverse target precision (as opposed to a polylogarithmic scaling in Ref. [14, 66, 67]), they can reach high enough precision in practice.

IK: TODO, discuss the structure of the paper.

II. SNAP COMPILATION

The SNAP protocol is designed for classical optimization tasks where there is a target unitary or state we are trying to approximate to error ϵ. For state preparation, we can consider having a tensor-network representation of a state $|\psi\rangle$; such a state should be the best classical approximation available. We assume that standard techniques in the literature have been used to obtain a Clifford+R_z state preparation circuit $|\psi(\boldsymbol{\theta})\rangle = V(\vec{\theta})|0\rangle$, with $\boldsymbol{\theta}$ representing the R_z angles in the circuit. That is, we have a mechanism to optimize our angles $\boldsymbol{\theta}$ so as to minimize the error $\epsilon = 1-|\langle\psi|\psi(\boldsymbol{\theta})\rangle|^2$. For time evolution [please explain how you define overlap; you can use trace for finite systems, not sure how you handle infinite; but I wouldn't mention infinite, just some idea of how to define error. cite that optimization of angles is possible] [Then emphasize that this idea is general, and will extend to not just different kinds of networks for stateprep/evolution, but other tasks as well.]

Typically, once the obtained Clifford+R_z circuit has error ϵ less than some error budget ϵ_b, the R_z gates are then independently synthesized into Clifford+T gates [refernce appendix/synthesis section]. The SNAP protocol instead asks: if synthesizing the angles $\boldsymbol{\theta}'$ is cheaper than synthesizing $\boldsymbol{\theta}$, can the change $\boldsymbol{\theta} \to \boldsymbol{\theta}'$ be made while maintaining $\epsilon(\boldsymbol{\theta}') < \epsilon_b$? We know that there are certain special rotations that are extremely cheap e.g. Clifford rotations [reference library/literature]. Hence, we consider the cost of just a single angle, asking the question: if the angle α is one of these special cheap angles, is there a replacement $\boldsymbol{\theta} \to \boldsymbol{\theta}'$ with $\theta'_i = \alpha$ satisfying $\epsilon(\boldsymbol{\theta}') < \epsilon_b$?

To address this question, we proceed in two steps. First, we construct a large library containing many cheap choices of α. Second, for each candidate snap $\theta_i \to \alpha$, we design a mechanism to efficiently estimate how effectively the remaining angles can be re-optimized to compensate for the resulting increase in infidelity. A SNAP may initially perturb the circuit substantially, but if the remaining variational degrees of freedom can absorb this perturbation and recover an infidelity below ϵ_b, then the replacement is accepted. Once an angle is snapped, its angle is then locked and never changed again; then the process repeats, asking the question regarding the remaining angles. The process continues until no snappable angles remain, at which point standard synthesis tools are used on the remaining angles.

The remainder of this section describes how such a library is generated, how the snap recoverability is estimated, and finally a complete description of the SNAP protocol.

A. Library

[haven't touched much in this subsection, do we plan to remove catalyst discussion? If so we can also remove the figure from this subsection. I want this section to be very untechnical, so very heuristic and intuitive explaination of how the library is generated, citing appendix/literature.]

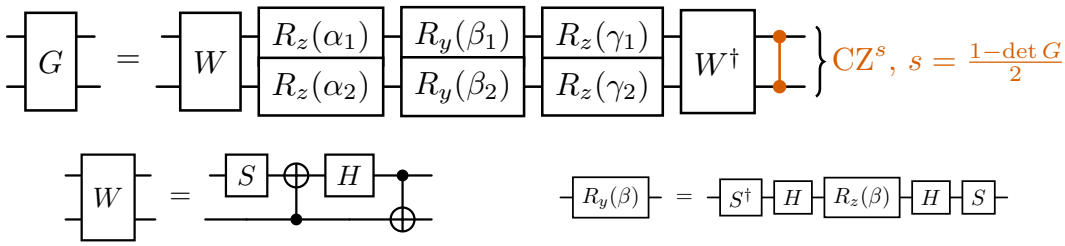
Snapping requires a set of angles at which an R_z rotation has an exact Clifford+T circuit, together with the cost of each circuit. Throughout this paper the rotation and the T gate are

$$R_z(\theta) = e^{-i\theta Z/2} = \begin{pmatrix} e^{-i\theta/2} & 0 \\ 0 & e^{i\theta/2} \end{pmatrix} \tag{1}$$

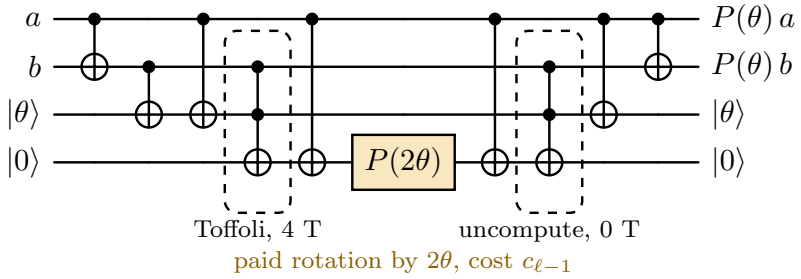
and

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = e^{i\pi/8}\, R_z(\pi/4) \tag{2}$$

We call such a set of angles α, each with the expected T cost $c(\alpha)$ of its circuit, a library. We build ours in



(a)



(b)

FIG. 1. Gate basis and library. (a) An $O(4)$ gate as six R_z rotations and Cliffords, Eqs. (B1) to (B3), together with the two identities it uses. W is the Clifford that maps the computational basis to the Bell basis, with phases chosen so that $W^\dagger(A_1 \otimes A_2)W$ is real orthogonal for every pair of $SU(2)$ matrices, and the four-gate circuit realizes it. The colored gate at the end is a Clifford, CZ, hence, it costs no T and is present only when $\det G = -1$, which is the CZ^s factor of Eq. (B1). (b) The doubling gadget behind catalysis [32], with $P(\theta) = \mathrm{diag}(1, e^{i\theta})$ and the catalyst $|\theta\rangle = (|0\rangle + e^{i\theta}|1\rangle)/\sqrt{2}$. Every gate is Clifford except two. The Toffoli writes the AND of its controls into a fresh $|0\rangle$ ancilla, which is Gidney's temporary AND [38] at 4 T rather than the 7 of a general Toffoli. It is then uncomputed by an X-basis measurement and a classically controlled CZ at no T cost. The yellow $P(2\theta)$ is the one library rotation the gadget consumes, a rotation by the doubled angle at the parent cost $c_{\ell-1}$. Two rotations by θ are produced, the catalyst is returned unchanged, and the ancilla returns to $|0\rangle$. we will probably want to split up this figure or remove it entirely. part a is specific to our stateprep approach, and belongs elsewhere, and part b I think is catalyst, which we plan to remove right?

two stages. The base library contains three kinds of angles. The Clifford angles, $\alpha \in (\pi/2)\mathbb{Z}$, cost zero, and the odd multiples of $\pi/4$ cost one T [68, 69], as discussed in Sec. A. The remaining angles are those that the circuit of Fig. 9 implements exactly [15, 70–72], built as described in Sec. A. We use the circuits with denominator exponent $k \leq 9$ and success probability $p \geq p_{\min} = 0.9$, Eq. (A6), and we did not vary $p_{\min}$. Each circuit costs the expected T count of Eq. (A5), with T_V from exact synthesis [68]. A circuit whose α_f is a multiple of $\pi/4$ may also take the fallback, and we charge it the cheaper of the two costs. This gives 2108 circuits whose angle is not a multiple of $\pi/4$, and 2002 of them always use the fallback because their α_f is not a multiple of $\pi/4$ either.

On $m = 1$ the fallback approximates its rotation to an operator-norm error ϵ_{fb}, which T_{fb} fixes through the fallback law of Sec. A. Each fallback therefore changes the unitary of the circuit by at most ϵ_{fb} in operator norm. Averaged over the measurement outcomes, the fallbacks add at most $\epsilon_{\mathrm{fb}}^2 \langle n_{\mathrm{fb}}^2 \rangle$ to the infidelity of the output with respect to the output with exact rotations, where n_{fb} is the number of fallbacks in one run of the circuit. We take the smallest T_{fb} among $18, 20, 22, \ldots$ for which this term is at most a tenth of the budget.

We add the negative of every angle and its shifts by multiples of $\pi/4$, since $R_z(-\alpha) = XR_z(\alpha)X$ and $R_z(\alpha + j\pi/4) = R_z(\alpha)\,T^j$ up to a global phase. A shift by an odd multiple of $\pi/4$ adds one T.

Catalysis [32] then adds angles by halving. The gadget of Fig. 1(b) consumes one Toffoli at 4 T [38] and one library rotation by 2θ, and it returns two rotations by θ together with the unchanged catalyst $|\theta\rangle$. Charging each output half of the gadget, a halved angle at level ℓ costs

$$c_\ell = 2 + \frac{c_{\ell-1}}{2} \tag{3}$$

where $c_{\ell-1}$ is the cost of its parent angle 2θ and c_0 is the cost of that angle in the base library. We apply three levels to every angle of the base library, including the multiples of $\pi/4$. The $\pi/8$, $\pi/16$ and $\pi/32$ families then cost 2.5, 3.25 and 3.625 T, respectively. The library holds 121 920 angles, and the expected distance from a uniformly random angle to the nearest one falls from 1.1×10^{-4} rad without catalysis to 2.7×10^{-5} rad. Each level moves the cost toward 4 T, the fixed point of Eq. (3), so further levels mainly add density.

Two conditions apply to every catalysis cost in this paper. First, Eq. (3) assumes that both outputs of the gadget are used, which requires two rotations by the same

angle, either in one circuit or across shots. If only one output is used, a level costs $4+c_{\ell-1}$. Second, each family of angles that shares a catalyst, which we call a fold class, needs one catalyst qubit per level, prepared once and held for the whole computation. A translation-invariant circuit has few fold classes, whereas a circuit with many distinct angles needs many catalyst qubits. Sun and Koczor [19] find that catalyst towers require less spacetime volume than conventional synthesis only up to code distances of about 13. We therefore report the number of fold classes together with the T counts. a lot of time is spent on catalysis, but this might not be necessary if we do cat0; I think I would prefer not to discuss catalysis in main text, leave it for appendix, or just a comment if we don't end up using it.

Rotations that never reach a library angle are synthesized at the end, at the precision that the remaining budget allows, and are charged the expected cost of the fallback protocol [15, 72], the fallback law of Sec. A at that precision. No rotation reaches this stage in the ground-state compilations of Sec. III A. For the taper circuits of Sec. III B we also report the cost when these rotations are synthesized with Ross-Selinger [14], and Sec. IV identifies the budget below which most rotations of the time-evolution circuits reach this stage. Last 3 sentences need a rewrite after sections mentioned are done I would remove this paragraph and simiply leave this detail for later subsection

B. Hessian-predicted SNAP recoverability

An essential component of the SNAP prototcol is the ability to estimate whether a SNAP $\theta_i \to \alpha$ is recoverable; that is, we wish to determine if there is a choice $\boldsymbol{\theta} \to \boldsymbol{\theta}'$ with $\theta'_i = \alpha$ satisfying $\epsilon(\boldsymbol{\theta}') < \epsilon_b$. Our approach to this problem is to evaluate it with a quadratic model, constructing the Hessian over the set $\mathcal{F}$ of free (unsnapped) angles:

$$H_{ij} = \frac{\partial^2 \epsilon}{\partial\theta_i\,\partial\theta_j}, \qquad i,j \in \mathcal{F}\,. \tag{4}$$

Since we assume we have the ability to optimize $\boldsymbol{\theta}$ so as to minimize $\epsilon(\boldsymbol{\theta})$, we can safely assume $\boldsymbol{\theta}$ is near a local minimum of ϵ with $\nabla\epsilon \approx 0$, hence motivating a quadratic model.

~~Let $\mathcal{F}$ be the set of free rotations. The re-optimization leaves the current angles $\boldsymbol{\theta}$ at a minimum of ε over the angles in $\mathcal{F}$, to the tolerance of the optimizer, with infidelity $\varepsilon_{\rm cur} = \varepsilon(\boldsymbol{\theta})$. We neglect the remaining gradient over $\mathcal{F}$ and write H for the Hessian over the free angles,~~

Consider snapping rotation $s \in \mathcal{F}$ to the library angle α, a displacement $\delta_s = \alpha - \theta_s$ taken modulo 2π in $(-\pi, \pi]$, and let $f = \mathcal{F} \setminus \{s\}$ label the remaining free angles. If their angles move by $\boldsymbol{\delta}_f$, the error rises to second order by

$$\Delta\epsilon = \tfrac{1}{2}\boldsymbol{\delta}^T H \boldsymbol{\delta} \tag{5}$$

$$= \boldsymbol{\delta}_f^T H_{ff} \boldsymbol{\delta}_f + 2\delta_s\, \boldsymbol{\delta}_f^T H_{fs} + H_{ss}\delta_s^2 \tag{6}$$

where $\boldsymbol{\delta} = (\boldsymbol{\delta}_f, \delta_s)$, and we have partitioned the Hessian by

$$H = \begin{pmatrix} H_{ff} & H_{fs} \\ H_{sf} & H_{ss} \end{pmatrix}. \tag{7}$$

To minimize the rise we take the derivative with respect to $\boldsymbol{\delta}_f$, giving the condition $H_{ff}\boldsymbol{\delta}_f + H_{fs}\delta_s = 0$. Hence the optimal $\boldsymbol{\delta}_f$ is given by

$$\boldsymbol{\delta}_f = -H_{ff}^{+} H_{fs}\, \delta_s, \tag{8}$$

where H_{ff}^{+} is the Moore-Penrose pseudoinverse of H_{ff} (which stands in for H_{ff}^{-1} because H_{ff} can be singular). Our Hessian-based SNAP recoverability approach thus estimates that snapping $\theta_i \to \alpha$ would result in an error increase of

$$\Delta\epsilon = \frac{\delta_s^2}{2}\left(H_{ss} - H_{sf}H_{ff}^{+}H_{fs}\right) \tag{9}$$

after all other free angles are re-optimized. It should be noted that once the Hessian is constructed, estimating the recoverability of many different SNAPs can be done very quickly; for example, for fixed cheap angle α, we can estimate $\Delta\epsilon$ corresponding to snapping $\theta_i \to \alpha$ for all angles θ_i in our circuit. Fig. ....

I propose we remove everything remaining in this section, it isn't necessary.

~~Without this response the rise would be $\frac{1}{2}H_{ss}\delta_s^2$, the quadratic term of Eq. (B7), so the fraction that survives is $r_s = \frac{\boldsymbol{\delta}^T H \boldsymbol{\delta}}{H_{ss}\,\delta_s^2} = 1 - \frac{H_{sf}\,H_{ff}^{+}H_{fs}}{H_{ss}}$ (10) with $H_{sf} = H_{fs}^T$. The product $r_s H_{ss} = H_{ss} - H_{sf}H_{ff}^{+}H_{fs}$ is the Schur complement of the block H_{ff} in H [73]. The Schur complement is the minimum of $\boldsymbol{\delta}^T H \boldsymbol{\delta}$ over $\mathbf{x}_f$ at $\delta_s = 1$ and lies between 0 and H_{ss}. The fraction r_s therefore lies between 0 and 1 and does not depend on δ_s. It equals 1 when $H_{fs} = 0$, so that no combination of the free angles reproduces the effect of moving θ_s, and it approaches zero when the free angles can replicate that effect, for example when two R_z rotations act on the same qubit with no gate between them. We take the size of the rise from the exact plug-in infidelity of Sec. B 2 and only the surviving fraction from Eq. (10), so the predicted infidelity after the free angles respond is $\varepsilon_{\rm pred}(s,\alpha) = \varepsilon_{\rm cur} + r_s\big[\varepsilon_{\rm init}(s,\alpha) - \varepsilon_{\rm cur}\big]$ (11) Figure 2 shows both parts for one rotation, the plug-in rise in panels (a) and (b) and the surviving fraction in panel (c).~~

C. SNAP protocol

The SNAP protocol takes as input a Clifford+R_z circuit, and assumes the following: i) the ability to optimize

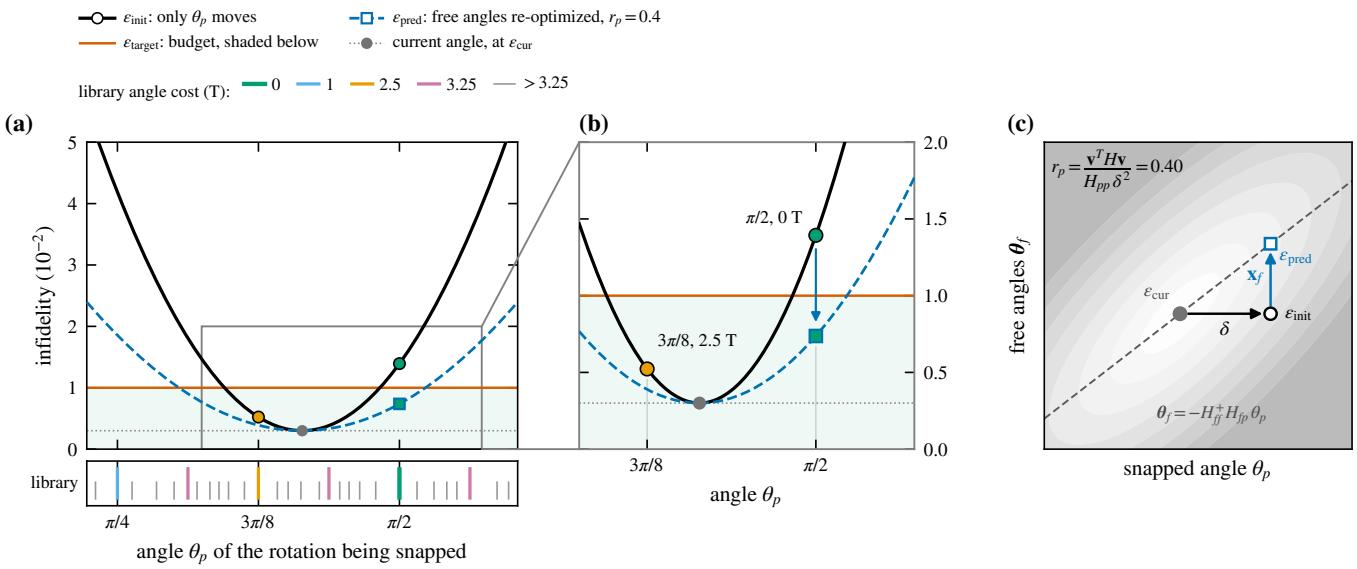


FIG. 2. Deciding one snap, with schematic values. (a) Infidelity of the circuit against the angle θ_p of one rotation p. The solid curve is the plug-in infidelity $\varepsilon_{\text{init}}$ of Eq. (B6), with every other angle held at its current value. The dashed curve is the prediction $\varepsilon_{\text{pred}}$ of Eq. (11) for the infidelity after the free angles re-optimize, here with $r_p = 0.4$. The gray dot marks the current angle, at the minimum of both curves, where the infidelity is ε_{cur} (dotted line). A snap fits the budget when the infidelity after the re-optimization lies in the shaded region below $\varepsilon_{\text{target}}$. The strip below the axis marks the library angles in this range, colored by their cost in T. The cheap angles are far apart and the expensive angles are dense. (b) Zoom of the boxed region in (a), with two candidate snaps. The circles are the plug-in infidelities of the two candidates and the square is the prediction for $\pi/2$. The angle $3\pi/8$ costs 2.5 T and fits the budget even if no other angle moves (orange circle). The Clifford angle $\pi/2$ costs no T, but with every other angle fixed it exceeds the budget (green circle). Once the free angles re-optimize, the infidelity at $\pi/2$ is predicted to fall below $\varepsilon_{\text{target}}$ (green square). The loop tries candidates in order of cost, so it tries $\pi/2$ first and accepts it if the re-optimized infidelity of the trial is at most $\varepsilon_{\text{target}}$. The rotation then costs no T, whereas a compiler that uses only the plug-in test would choose $3\pi/8$ at 2.5 T. (c) The quadratic model behind Eqs. (8) and (10). The horizontal axis is the snapped angle θ_p and the vertical axis is the free angles $\boldsymbol{\theta}_f$, drawn along a single direction, both measured from their current values. The shading shows the rise $\frac{1}{2}\mathbf{v}^T H \mathbf{v}$, lighter where it is smaller. The valley is tilted because $H_{fp} \neq 0$, so the free angles can partly compensate a change of θ_p. The dashed line $\boldsymbol{\theta}_f = -H_{ff}^{+} H_{fp}\,\theta_p$ is the floor of the valley, the best free angles for each θ_p. The snap moves θ_p by δ with the free angles fixed, which raises the infidelity from ε_{cur} (gray dot) to $\varepsilon_{\text{init}}$ (circle). The free angles then move by $\mathbf{x}_f$ to the floor, which lowers it to $\varepsilon_{\text{pred}}$ (square). The surviving fraction is $r_p = (\varepsilon_{\text{pred}} - \varepsilon_{\text{cur}})/(\varepsilon_{\text{init}} - \varepsilon_{\text{cur}}) = 0.40$. Without the tilt the floor would be the horizontal axis, with $\mathbf{x}_f = 0$ and $r_p = 1$. some of this notation needs to be fixed, lets remove the r_s factor, it is overcomplicating the issue.

the angles $\boldsymbol{\theta}$ so as to minimize the error ϵ of the circuit, ii) a library of cheap angles, and iii) the ability to construct the Hessian $H = d^2\epsilon/d\theta_i d\theta_j$. The previous subsections explained how such a library can be created and how the Hessian can be used to quickly estimate if a SNAP $\theta_s \to \alpha$ is recoverable below budget error ϵ_b. We now describe the complete SNAP protocol, illustrated in 3. The key thing to appreciate is that while optimizing $\boldsymbol{\theta}$ to minimize ϵ is classically tractable, doing so for each angle α in the library and each free angle θ_i in the circuit would be prohibitively expensive. Constructing the Hessian is similar, in that is classically tractable to do. However, once the Hessian is constructed, the impact of a SNAP $\theta_s \to \alpha$ can be estimated extremely quickly, and the same Hessian can be used to quickly test all (θ_i, α) pairs. So, while the Hessian is an estimation of the recoverability of a SNAP, it offers two advantages; i) for a fixed SNAP $\theta_s \to \alpha$, the Hessian tells us how to optimally adjust the remaining free angles $\mathcal{F} \setminus \{s\}$, and ii) the Hessian can be reused to test all possible SNAPs, and hence only needs to be computed once (per accepted SNAP). Recognizing that the Hessian just provides an estimate in a quadratic model, we do not accept proposed SNAPs automatically. If the Hessian predicts a SNAP $\theta_s \to \alpha$ is recoverable, we test by snapping, adjusting the free angles to and reoptimizing the remaining angles. Only if the reoptimized circuit actually achieves $\epsilon < \epsilon_b$ do we accept the SNAP; otherwise this SNAP is rejected.

The first step of the SNAP protocol accepts a Clifford+R_z circuit with a set of free angles $\mathcal{F}$, and optimizes the angles $\boldsymbol{\theta}$. The library contains a list of cheap angles α with corresponding T-gate cost $T(\alpha)$. The idea is to prioritize SNAPs with low $T(\alpha)$. So the Hessian is constructed, and used to estimate if a snap $\theta_i \to \alpha$ is recoverable for any of the free angles θ_i in the circuit, with $T(\alpha) = 0$, i.e. it can be snapped to a Clifford. If no such SNAP exists, we allow the cost to be raised e.g. to $T(\alpha) = 1$. However, if $\theta_s \to \alpha$ is predicted to be recov-

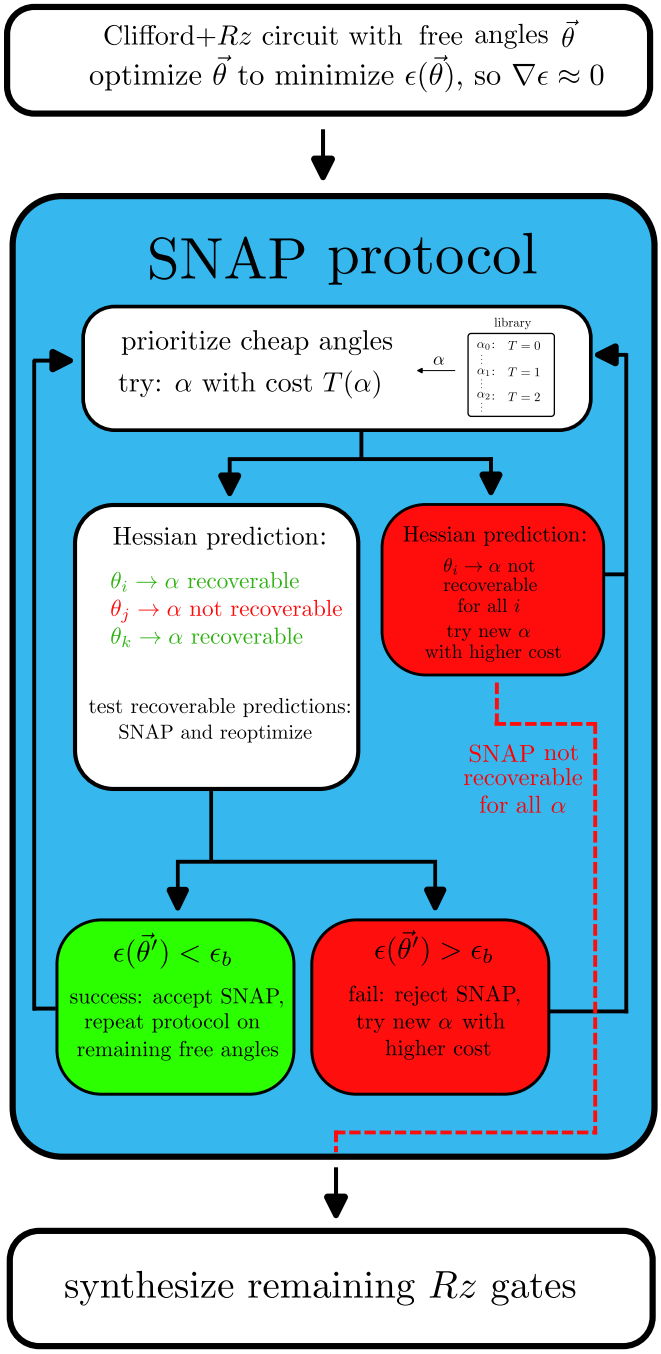


FIG. 3. add caption feel free to make suggestions

erable, we test if this is true. This is done by modifying the circuit angles $\boldsymbol{\theta} \to \boldsymbol{\theta}' = \boldsymbol{\theta} + \boldsymbol{\delta}$, and then reoptimizing the remaining free angles $\boldsymbol{\theta}_f$ so as to minimize ϵ. If all predicted SNAPs fail to achieve their predicted $\epsilon < \epsilon_b$, we allow the cost $T(\alpha)$ to increase. However, if a predicted SNAP succeeds, the SNAP is accepted, and the whole process repeats, but with one fewer free angle. The SNAP protocol only ends when every α in the library is predicted to fail for every free angle θ_i. If that occurs, then every remaining free angle is then synthesized in the standard approach [refer...].

III. STATE PREPARATION

in here I propose we describe essential details of how we define an iMPS, infidelity of two iMPS, and how we optimize Clifford+Rz for infid using environments; O(4) details can go in appendix. Then I will describe how we construct Hessian using overlaps. Then I will give stateprep results. so four subsections: object/error definition, optimization of angles, Hessian construction, results.

A. XXZ ground state

B. Fixed circuits at tight budgets

IV. TIME EVOLUTION

We now compile the time-evolution operator e^{-iHt} of a spin chain. The procedure is the one of Sec. II. We fix a Clifford+R_z circuit and an error $\epsilon(\boldsymbol{\theta})$, optimize the angles $\boldsymbol{\theta}$, and then snap them onto the library one at a time, with the Hessian of ϵ deciding which snaps to try first. Two things change from state preparation. The target is a unitary, so the error has to be small for every input and not just for one state. The circuit is also translation invariant, so each angle is shared by many rotations and a single snap fixes all of them.

We compare every compiled circuit with product formulas that use the same gates, the same synthesis protocol and the same definition of the error. At an error of 10^{-3} per site, SNAP needs 3.3 to 3.4 times fewer T gates than the cheapest product formula when the chain is in a transverse field, and 1.3 to 2.1 times fewer without one, for times up to $t = 4$. Over every error the compiled circuits reach so far, the factor is 1.1 to 4.8 with the field and 1.1 to 3.9 without it. Product formulas whose number of steps is chosen from a rigorous error bound need 8 to 51 times more T gates than SNAP.

A. Circuit and error

We take the XXZ chain in a transverse field,

$$H = \sum_i \left(S_i^x S_{i+1}^x + S_i^y S_{i+1}^y + \Delta\, S_i^z S_{i+1}^z\right) + h_x \sum_i S_i^x, \quad (12)$$

with $S_i^\alpha = \sigma_i^\alpha/2$ and $\Delta = 0.25$, at two fields, $h_x = 1.6$ and $h_x = 0$. Times are $t = 1$, 2 and 4 in units of the inverse exchange coupling. Without the field a product formula needs no field layer, which makes $h_x = 0$ the easier case for the baselines of Sec. IV D. Add $t = 6$ when Isaac's run comes back.

The circuit is a brickwork of D sublayers, shown in Fig. 4(a). Sublayer ℓ applies one two-qubit gate $G(\boldsymbol{\theta}_\ell)$ to every bond of one parity, the even bonds when ℓ is even and the odd bonds when ℓ is odd. For $h_x = 1.6$ the gate is

$$G(\boldsymbol{\theta}_\ell) = e^{-i(a\,XX + b\,YY + c\,ZZ)/2}\left[R_x(e) \otimes R_x(f)\right] \quad (13)$$

with $\boldsymbol{\theta}_\ell = (a, b, c, e, f)$. For $h_x = 0$ we replace the single-qubit factor by $R_z(d) \otimes R_z(-d)$, so that $\boldsymbol{\theta}_\ell = (a, b, c, d)$. We add this staggered rotation because the zero-field circuits train poorly at large depth without it. Each factor in Eq. (13) is one R_z rotation between Cliffords. For example, $e^{-icZZ/2} = \mathrm{CNOT}\,[I \otimes R_z(c)]\,\mathrm{CNOT}$, and the XX, YY and R_x factors follow from it by single-qubit Cliffords, as in Fig. 4(b). Every bond therefore carries five rotations, and a circuit of depth D has $5D/2$ rotations per qubit.

All bonds of a sublayer share its angles. The circuit is then invariant under translation by two sites and is defined on any even number of sites, and it has only $5D$ bulk angles for $h_x = 1.6$ and $4D$ for $h_x = 0$, whatever the size of the system. The first-order product formula is one point of this family. Two sublayers with $a = b = \tau/2$, $c = \Delta\tau/2$ and $e = f = h_x\tau/2$ apply one Trotter step of length τ.

The compiled circuit U has to approximate e^{-iHt} on every input. We test it on product states $|\phi_k\rangle$ whose single-qubit factors are drawn uniformly from the Bloch sphere. Such inputs are cheap to evolve with tensor networks, and a unitary learned from enough low-entanglement inputs generalizes to all inputs [62, 74]. For input k the fidelity is $F_k = |\langle\phi_k|e^{iHt}\,U|\phi_k\rangle|^2$. With a fixed error per site, F_k decays exponentially in the number of sites N, so we report the infidelity per site,

$$\epsilon = 1 - \frac{1}{K}\sum_{k=1}^{K} F_k^{1/N}, \quad (14)$$

averaged over K inputs. This is the local infidelity of Ref. [62] on a finite chain. Throughout this section ϵ means Eq. (14) and ϵ_b is the budget on it. The fidelity of the whole chain is about $(1-\epsilon)^N$, so $\epsilon = 10^{-4}$ on $N = 100$ sites leaves a fidelity of 0.99.

We train, snap and evaluate on an open chain of $N = 100$ sites, with every state held as a matrix product state [75]. The training error uses $K = 12$ inputs. The ϵ we report uses $K = 8$ other inputs, so it is a held-out value. The references $e^{-iHt}|\phi_k\rangle$ come from a fourth-order product formula with a step small enough that its own error is negligible. We use an open chain rather than a ring because random product inputs build up entanglement linearly in t, and on a ring every bipartition cuts two bonds, which roughly squares the bond dimension. On the odd sublayers the two end sites have no partner, so each gets its own gate $R_zR_yR_z$ there, six angles per odd sublayer, trained together with the bulk angles. An end gate appears once per circuit, while each bulk gate appears $N/2$ times, so the end gates add at most 2.5% to the T count. For every compiled circuit the held-out ϵ is between 0.92 and 1.05 times the training value, with a median ratio of 1.00. This paragraph is the only part of the section that depends on the system-size question raised on 22 Oct. With an infinite chain, Eq. (14) becomes the leading eigenvalue of the mixed transfer matrix [62], the end gates go away, and this paragraph shrinks to two sentences.

Placeholder for Fig. 4.
(a) brickwork of D sublayers on a chain, one gate $G(\boldsymbol{\theta}_\ell)$ per bond,
end gates on the odd sublayers
(b) $G(\boldsymbol{\theta}_\ell)$ as five R_z rotations between Cliffords
(c) one stretch of a compiled circuit, rotations colored by T cost:
product formula with every rotation synthesized,
against SNAP

FIG. 4. The time-evolution circuit. (a) Each sublayer applies the gate $G(\boldsymbol{\theta}_\ell)$ of Eq. (13) to every bond of one parity, with the same angles on every bond, so the circuit is invariant under translation by two sites. On the odd sublayers each end site gets its own single-qubit gate. (b) The gate written as five R_z rotations between Cliffords. The rotations by a, b and c act on the parity of the two qubits, and those by e and f on each qubit. (c) A stretch of the circuit at $h_x = 1.6$, $t = 4$ and $\epsilon = 10^{-3}$, with each rotation colored by its T cost, for the cheapest product formula (top) and for SNAP (bottom). Panel (c) follows David's suggestion on Slack (11 Oct), a colorbar for the T cost as in the tQPE paper.

B. Optimizing the angles

For each field and time we train circuits of increasing depth. Each depth starts from three points: the first-order product formula of that depth, the optimum of the previous depth with identity sublayers appended, and the product formula with 0.1 rad of noise on its angles. From each start we minimize the training error by trust-region Newton steps. The Hessian comes from central differences of exact gradients, and each one serves several steps. A run stops when ϵ falls by less than a part in 10^6 per step or reaches the truncation noise of the matrix product states, with no cap on the number of iterations. We keep the best of the three runs. Its error $\epsilon_0(D)$ is the floor of that depth, the error of the circuit before any angle is snapped. We stop adding depth once the floor falls by less than a factor of 1.5 over eight sublayers, or reaches 10^{-10}. Figure 5 shows the floors. The depth needed for a given floor grows roughly linearly with t. This is the variational compression of Refs. [55, 56, 60, 62], and it supplies the Clifford+R_z circuit and the optimizer that

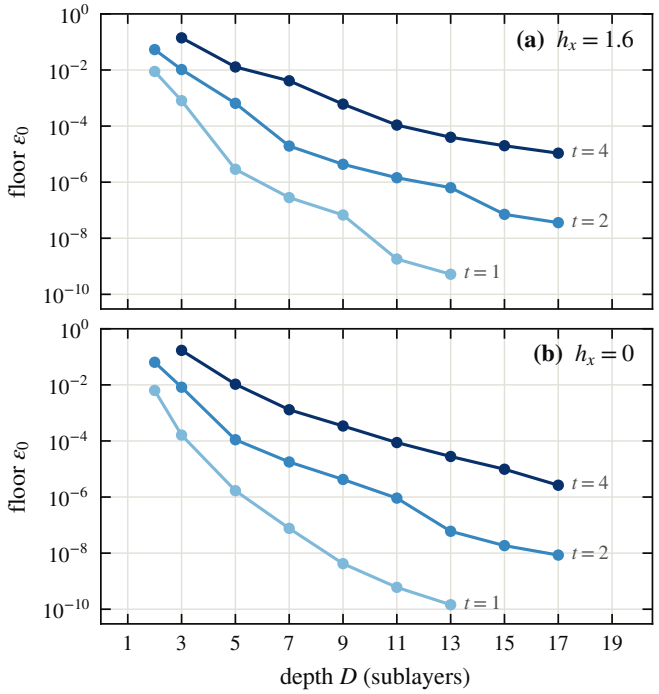


FIG. 5. Floors of the trained circuits. The training error ϵ_0 of the best circuit at each depth D, before any snap, on the 100-site chain, for $t = 1$, 2 and 4 (light to dark) at $h_x = 1.6$ (a) and $h_x = 0$ (b). The ladders at $t = 2$ and 4 are still being extended. Could move to the appendix if space is short.

SNAP assumes.

C. Snapping shared angles

The compiler is the loop of Sec. II, with three changes.

First, an angle of sublayer ℓ sets the rotation on every bond of that sublayer, $N/2$ bonds for even ℓ and $N/2-1$ for odd ℓ. Snapping it to a library angle α costs $T(\alpha)$ per bond, about $T(\alpha)/2$ T gates per qubit. Every bulk angle has nearly the same multiplicity, so the loop still tries candidates in order of $T(\alpha)$. The end angles are re-optimized with the bulk after every snap. When the loop stops they are moved to Clifford or library angles where the budget allows, and the rest are synthesized (App. C).

Second, the error is no longer a sinusoid in each angle. An angle shared by m rotations enters each overlap as a trigonometric polynomial of degree m, Eq. (B11). We fix that polynomial exactly from the overlap at equally spaced values of the angle, about 50 of them for most angles at $N = 100$, and it then gives the plug-in error at every library angle. We compute the Hessian of the training error over the free bulk and end angles from central differences of the exact gradient, Eq. (B16), and recompute it after every accepted snap. After each trial snap the free angles are re-optimized by the same trust-region Newton method used in training, again with no cap on the number of iterations.

Third, near the floor the loop has to leave room for the angles it does not snap. When ϵ_b is only a few times ϵ_0, the loop could spend the whole budget on snaps and leave the remaining angles to be synthesized at a precision that costs more than the snaps saved. We therefore accept a snap only if the budget it leaves can still pay for synthesizing the rest, and we report the cheapest prefix of the accepted sequence, Eq. (B18).

We compile budgets in half decades from $\epsilon_b = 10^{-1}$ to 10^{-9}, where the budget applies to the training error. Each budget is compiled at every trained depth whose floor is at most $\epsilon_b/1.5$, and the figures show the cheapest result at each error. Compiles at the tighter budgets are still running. So far the points reach $\epsilon \approx 10^{-6}$ at $t = 1$ with the field and 10^{-9} without it, 10^{-5} at $t = 2$ and 3×10^{-4} at $t = 4$. Snapshot of 22 Oct, 01:20 PDT.

When the loop stops, every rotation is one of three kinds. A rotation at a multiple of $\pi/4$ is exact and costs 0 or 1 T. A rotation at another library angle is exact on the success branch of its circuit and costs the expected T count of Eq. (A5). A rotation that was never snapped is synthesized with the non-mixed fallback protocol of Ref. [15], at the precision that the remaining budget allows. We use no catalysis and no mixed synthesis anywhere in this section. A synthesized rotation takes its fallback branch with probability at most 0.1, and we draw these branches at random in the $N = 100$ simulation, so the reported ϵ includes them. A library circuit whose failure cannot be undone exactly is corrected by a synthesized rotation of known precision, and we bound the effect of these corrections and add it to ϵ (App. C). The draws for the SNAP points have not run yet. Until they do, every SNAP point in the figures and Table I sits at its fallback bound, which can only overstate its error. The bound moves ϵ by less than 1% for 90% of the points.

D. Baselines

We compare against product formulas that use the same synthesis protocol, with their error obtained in two ways.

The first measures it. We split H into the even bonds, the odd bonds and the field, and build formulas of order 1, 2 and 4 [21] with r steps. Every distinct angle is synthesized once at precision ϵ_{syn} with the protocol of Ref. [15], and all its copies use the same circuit, so their synthesis errors add coherently. We scan the order, r and ϵ_{syn}, measure ϵ for each formula exactly as for SNAP, on the same held-out inputs at $N = 100$ with the fallbacks drawn, and keep the cheapest formula at each error. This baseline relies on the same classical simulation as SNAP, and it is the strongest version of the conventional approach that we know how to build. The orderings and field placements of Liu et al. [76] at orders 2, 4 and 6, measured on the same inputs, fall close to the same curve.

The second takes the error from a bound, as one would

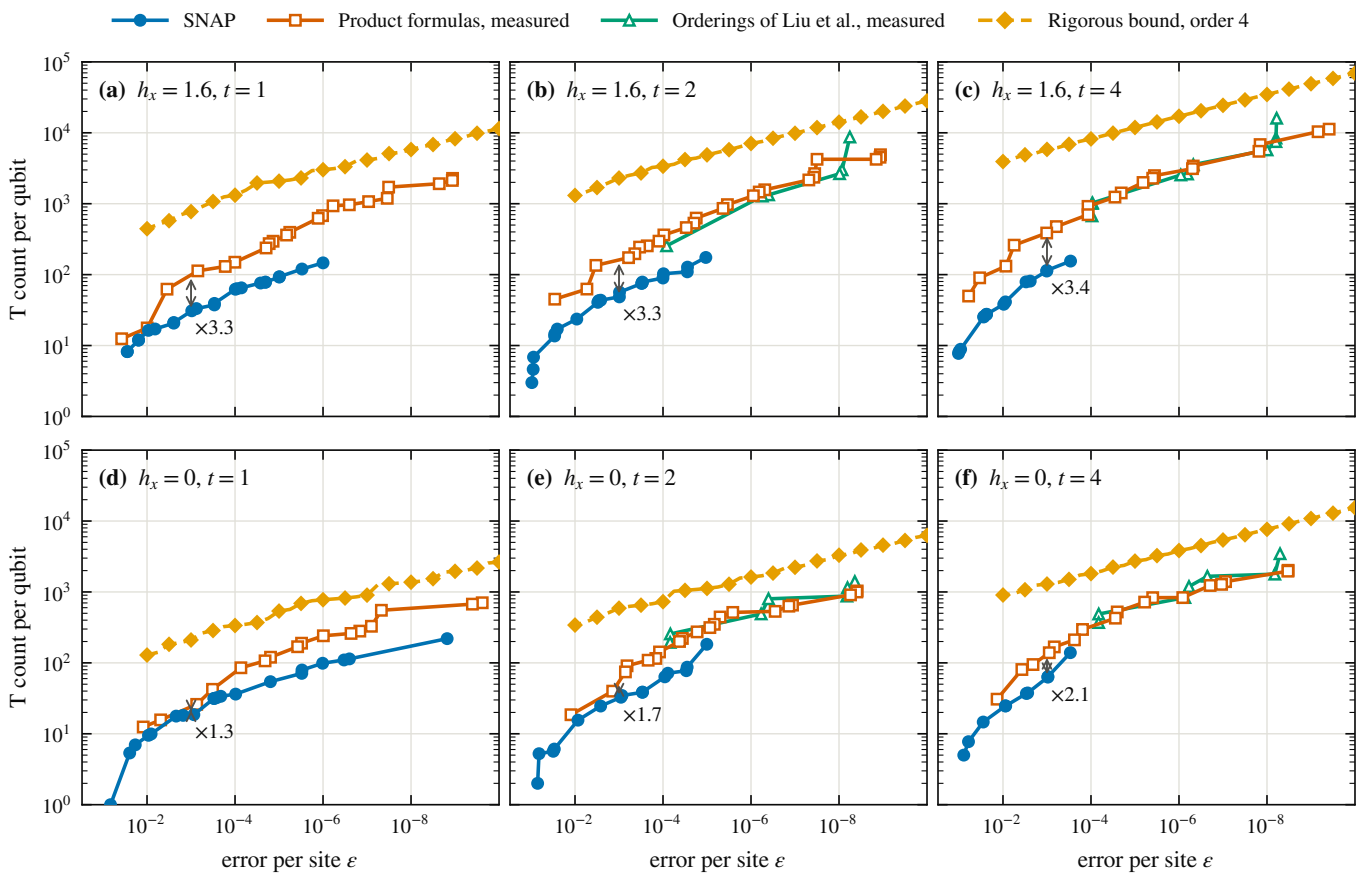


FIG. 6. T count per qubit against the error per site ϵ, Eq. (14), measured on 100 sites, with the error decreasing to the right. (a–c) $h_x = 1.6$ and (d–f) $h_x = 0$, at $t = 1$, 2 and 4. Circles are SNAP. Squares are the product formulas of order 1, 2 and 4 with measured error and the fallback branches drawn. Triangles are the orderings and field placements of Ref. [76] at orders 2, 4 and 6, measured on the success branch at $t = 2$ and 4. For our product formulas at these times, drawing the fallbacks raises ϵ by a median of 2%. Diamonds are fourth-order product formulas with the number of steps and the synthesis precision chosen from the bound of Ref. [76], plotted at their guaranteed ϵ. Each series is its lower envelope, the cheapest circuit at or below each ϵ. The arrows give the ratio of the product formula to SNAP at $\epsilon = 10^{-3}$, as in Table I. SNAP points sit at their fallback bound until their draws run, Sec. IV C.

without a classical simulation. We use the ordering-aware Trotter bound of Ref. [76], which is within a factor of 1.5 of the exact Trotter error for these formulas on 6 to 10 sites. We choose r so that the bound, converted to a guarantee on Eq. (14) at $N = 100$, meets the target, and we synthesize every rotation at an equal share of what remains. Fourth-order brickwork formulas are the cheapest at every time and error we tried. The error of this baseline is guaranteed rather than measured.

Isaac's third comparison, the trained circuit with every rotation synthesized and no snaps, has not been run. It separates the gain from compression from the gain from snapping. Sec. IV E estimates it from rotation counts for now.

E. Results

Figure 6 shows the T count per qubit against ϵ at $t = 1$, 2 and 4. Table I gives the ratio of the T count of the measured product formula to that of SNAP at three errors and every time. SNAP is cheaper than every product formula at every error reached so far. With the field the factor is 3.3 to 3.4 at $\epsilon = 10^{-3}$ for all three times and 1.1 to 4.8 over all errors. At $t = 1$, where the compiles reach furthest, it rises to 4.7 at $\epsilon = 10^{-6}$. Without the field the factor is 1.1 to 3.9, and largest at $t = 1$ near $\epsilon = 4 \times 10^{-8}$. At $\epsilon = 10^{-3}$ it grows with time, from 1.3 at $t = 1$ to 2.1 at $t = 4$. The gap is smallest at the loosest errors. At $t = 1$ it is a factor of 1.1 near $\epsilon = 10^{-2}$ with the field and near 2×10^{-3} without it.

All of the saving comes from the cost of each rotation. Between $\epsilon = 10^{-2}$ and 10^{-5} a SNAP circuit spends 0.3 to 5.6 T gates per rotation, while the cheapest product formula spends 2.7 to 16. The trained circuits do not

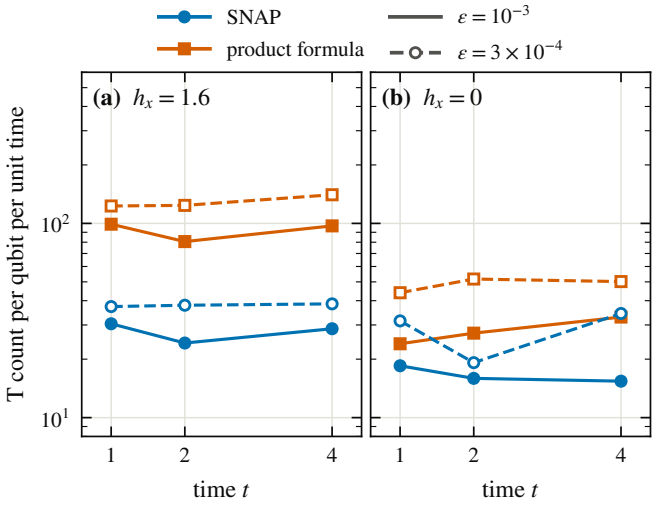


FIG. 7. T count per qubit per unit time against t at fixed error per site, for SNAP (circles) and the cheapest measured product formula (squares), at $\epsilon = 10^{-3}$ (solid) and 3×10^{-4} (dashed), for $h_x = 1.6$ (a) and $h_x = 0$ (b). Both are read off the lower envelopes of Fig. 6.

have fewer rotations. With the field they have 0.8 to 1.9 times as many as the cheapest product formula at the same error, and without it 1.2 to 6 times as many, because a zero-field product formula has three rotations per bond instead of five and needs few steps at these times. Compression alone, with every rotation synthesized as in the product formula, would therefore save little or nothing here. Replace the last sentence with the measured comparison once it is run. The share of angles that end at Clifford angles falls with time and with the budget. At $\epsilon_b = 10^{-3}$ it is 64%, 40% and 10% at $t = 1$, 2 and 4 with the field, and 71%, 59% and 41% without it. Most of the rest end at library angles. None of the 174 circuits compiled at $\epsilon_b \geq 10^{-4}$ leaves an angle unsnapped. Below that, 10 of the 34 circuits do, up to 46% of the angles in one of them.

The rigorous baseline is far more expensive. Between $\epsilon = 10^{-3}$ and 10^{-6} it needs 21 to 51 times more T gates than SNAP with the field and 8 to 21 times more without it. Most of this gap is the price of using a bound. The same formulas with measured error are already 3 to 15 times cheaper than with the bound. SNAP and the measured baseline both avoid that price because a classical simulation of the circuit is available, which is the setting this paper assumes.

F. Cost per unit time

Figure 7 shows the T count per qubit and per unit time. Up to $t = 4$ it changes little for SNAP. At $\epsilon = 10^{-3}$ SNAP spends 24 to 30 T gates per qubit per unit time with the field and 15 to 19 without it. The product formula needs 81 to 99 with the field, and without it rises from 24 at $t = 1$ to 33 at $t = 4$. At $\epsilon = 3 \times 10^{-4}$ SNAP spends 37 to 39 with the field, against 123 to 140 for the product formula. For the whole chain at $t = 4$ and $\epsilon = 3 \times 10^{-4}$, where the 100 sites keep a fidelity of 0.97, SNAP needs 1.5×10^4 T gates and the product formula 5.6×10^4. $t = 6$ will show whether the gap opens with time.

TABLE I. Ratio of the T count of the cheapest measured product formula to that of SNAP, at equal error per site ϵ on 100 sites, with the fallback branches of the product formulas drawn. SNAP points whose fallback branches are not drawn yet sit at their fallback bound, which can only overstate their error. A dash marks a time at which the compiled circuits do not yet reach that ϵ.

	$h_x = 1.6$			$h_x = 0$		
t	10^{-2}	10^{-3}	10^{-4}	10^{-2}	10^{-3}	10^{-4}
1	1.1	3.3	2.4	1.4	1.3	2.0
2	2.4	3.3	3.9	1.4	1.7	2.5
4	3.4	3.4	–	1.7	2.1	–

G. Worst-case error

The error per site is an average over inputs. A guarantee for every input needs the operator norm of $U - e^{-iHt}$, minimized over a global phase, which grows with N and roughly with $\sqrt{\epsilon}$. Measure this relation on the open-chain circuits once the runs are done, and add the operator-norm version of Fig. 6 that Isaac asked for.

V. END-TO-END ANALYSIS

Propose DSF simulation on e.g. 100 qubits and show something is m̃egaquap with our techniques (hopefully)

VI. DISCUSSION

Alternatively, if considering a ground state preparation problem, and one has the ability to optimize the energy one can consider defining the error as the energy error between the circuit and the tensor [fix later]

Appendix A: Rotation Synthesis: Review

In this section we review the cost of a single-qubit rotation $R_z(\theta) = e^{-i\theta Z/2}$ in the Clifford+T gate set, ending with the circuits from which Sec. II A builds its library. Figure 8 shows how SNAP departs from synthesizing each rotation on its own.

Kliuchnikov, Maslov and Mosca [68] showed that a single-qubit unitary admits an exact Clifford+T circuit if and only if its entries lie in the ring $\mathbb{Z}[1/\sqrt{2}, i]$. Writing

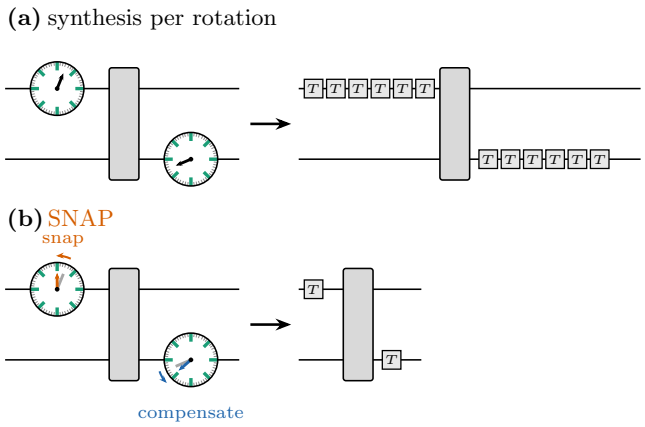


FIG. 8. Synthesis per rotation and SNAP. Each dial is an R_z rotation, its needle is the angle, and the gray box is a two-qubit gate. The green notches mark the few angles that Clifford+T implements exactly at a low cost, and the gray ticks mark the many exact angles that cost more. (a) Synthesizing each rotation on its own to a small error $\epsilon_{\rm rot}$ replaces it with a long Clifford+T sequence, drawn as a string of T gates. (b) SNAP turns one needle onto a green notch (orange arrow), and the other angles re-optimize to compensate (blue arrow). Repeating this until the budget $\epsilon_{\rm tot}$ is spent leaves most rotations at cheap exact angles, with a few T gates each. lets put this in the introduction as a cartoon explanation of what we are doing. I am worried that in (b) both angles are shown to be low-T, but only the snapped one should be low-T. Maybe we change it and just emphasize in the caption that here there is only a single SNAP taking place, with a single recovery angle; in practice there will be many recovery angles, and many snaps

$\omega = e^{i\pi/4}$, every such unitary is, up to a global phase,

$$U = \frac{1}{\sqrt{2}^{\,k}} \begin{pmatrix} u & -v^* \\ v & u^* \end{pmatrix}, \qquad u, v \in \mathbb{Z}[\omega], \tag{A1}$$

with $|u|^2 + |v|^2 = 2^k$, where k is the smallest denominator exponent for which such u and v exist. Their exact synthesis algorithm removes one factor of $\sqrt{2}$ from the denominator of $|u|^2$ at each step, using at most one T gate per step, so the T count of U is at most $2k$ [68]. For R_z rotations the exactly implementable angles are the multiples of $\pi/4$. The even multiples are Clifford and cost nothing, and the odd multiples cost one T [68].

A generic $R_z(\theta)$ must be approximated to some operator-norm error ϵ. Without ancillas, the algorithm of Ross and Selinger [14] returns a circuit with

$$T_{\rm RS}(\epsilon) = 3\log_2(1/\epsilon) + O\big(\log\log(1/\epsilon)\big) \tag{A2}$$

T gates for a typical angle. Given a factoring oracle, this is the shortest ancilla-free circuit, and without one it is near-optimal under a number-theoretic hypothesis [14]. On the angle sets used in this paper, the pygridsynth implementation [77] returns $T = 3.05\log_2(1/\epsilon) + 1.9$ to within half a T gate.

An ancilla and a measurement lower the cost [15, 70–72]. Given u, v and k as in Eq. (A1), the two-qubit circuit V_u of Fig. 9 applies U between two CNOTs that are controlled by the target qubit, so it applies U to an ancilla in $|0\rangle$ when the target is in $|0\rangle$ and XUX when it is in $|1\rangle$. The CNOTs are Clifford, so the T count T_V of V_u is that of U. We take T_V from the exact synthesis algorithm of Ref. [68], which uses the minimal number of T gates, and for every circuit we use it is $2k-2$ or $2k$. Measuring the ancilla with outcome m applies to the target the operator

$$\begin{aligned} \langle 0|V_u|0\rangle &= \frac{1}{\sqrt{2}^{\,k}} \begin{pmatrix} u & 0 \\ 0 & u^* \end{pmatrix} = \sqrt{p}\, R_z(\alpha) \\ \langle 1|V_u|0\rangle &= \frac{1}{\sqrt{2}^{\,k}} \begin{pmatrix} v & 0 \\ 0 & -v^* \end{pmatrix} = i\sqrt{1-p}\, R_z(\alpha_f) \end{aligned} \tag{A3}$$

where $\langle m|$ and $|0\rangle$ act on the ancilla, and

$$p = \frac{|u|^2}{2^k}, \qquad \alpha = -2\arg u, \qquad \alpha_f = \pi - 2\arg v \tag{A4}$$

On $m=0$, which occurs with probability p, the target receives $R_z(\alpha)$ exactly. On $m=1$ it receives $R_z(\alpha_f)$ up to a global phase. If α_f is a multiple of $\pi/4$, we undo it with $R_z(-\alpha_f)$ at $T_{\rm corr} = 0$ T for an even multiple and 1 T for an odd one, and apply V_u again with a fresh ancilla until $m=0$. This is repeat-until-success [70, 71]. Otherwise we apply a Clifford+T approximation $\widetilde{R}_z(\alpha - \alpha_f)$ of the remaining rotation once, at an expected $T_{\rm fb}$ T, which is the fallback of Ref. [72]. The expected cost of the rotation is

$$\mathbb{E}[T] = \begin{cases} \dfrac{T_V + (1-p)\,T_{\rm corr}}{p}, & \alpha_f \in (\pi/4)\mathbb{Z} \\ T_V + (1-p)\,T_{\rm fb}, & \text{otherwise} \end{cases} \tag{A5}$$

Refs. [15, 71, 72] use circuits of this kind to approximate an arbitrary angle to error ϵ, and their costs are empirical fits over random angles. Repeat-until-success costs $1.15\log_2(1/\epsilon)+9.2$ T [71], the fallback costs $1.03\log_2(1/\epsilon)+5.75$ T, and a random mixture of two fallback circuits costs $0.53\log_2(1/\epsilon)+4.86$ T [15].

SNAP uses these circuits in the opposite direction. Instead of choosing u to approximate a given angle, we collect the angles α that the circuits implement exactly and move rotations onto them. By Eq. (A4), the phase of u sets the angle and the magnitude of u sets the success probability. Since $|u|^2+|v|^2 = 2^k$, the failure probability is $1-p = |v|^2/2^k$, so a large $|u|$ and a small $|v|$ give a circuit that rarely fails. A circuit succeeds with probability at least $p_{\rm min}$ when

$$|u|^2 \geq p_{\rm min}\, 2^k \tag{A6}$$

which is the condition on u used in Ref. [15]. At fixed k, each $u \in \mathbb{Z}[\omega]$ that satisfies Eq. (A6), and for which some $v \in \mathbb{Z}[\omega]$ has $|v|^2 = 2^k - |u|^2$, gives one circuit.

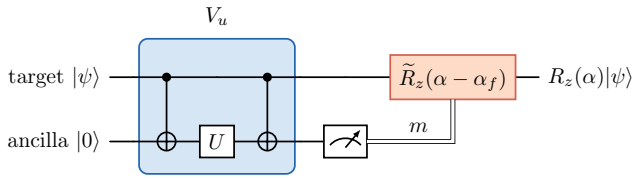


FIG. 9. Probabilistic synthesis of $R_z(\alpha)$ with fallback [15, 72]. The blue box V_u is the single-qubit unitary U of Eq. (A1) on the ancilla between two CNOTs controlled by the target, so the ancilla receives U when the target is in $|0\rangle$ and XUX when it is in $|1\rangle$. On the outcome $m = 0$, which occurs with probability $p = |u|^2/2^k$, the target has received $R_z(\alpha)$ exactly. On $m = 1$ it has received $R_z(\alpha_f)$ up to a global phase, and the orange box applies $\widetilde{R}_z(\alpha - \alpha_f)$, a Clifford+T approximation of $R_z(\alpha - \alpha_f)$. When α_f is a multiple of $\pi/4$, the orange box is instead the exact correction $R_z(-\alpha_f)$, followed by another application of V_u.

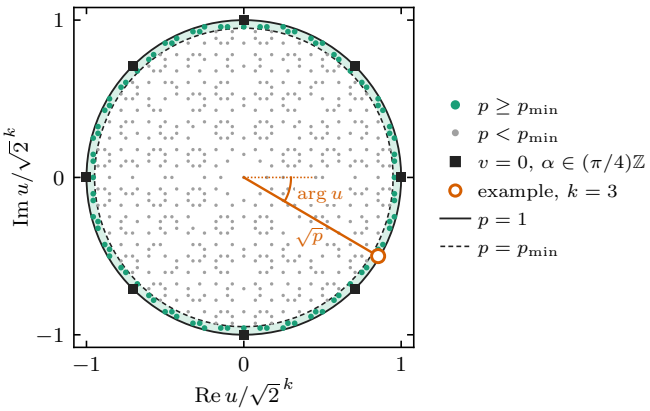


FIG. 10. Circuits of Fig. 9 with denominator exponent $k \leq 4$, drawn at $u/\sqrt{2}^k$ in the complex plane. Each dot is a $u \in \mathbb{Z}[\omega]$ for which some $v \in \mathbb{Z}[\omega]$ completes the unitary U of Eq. (A1). The orange circle is the example in the text, and the orange line marks its distance $\sqrt{p}$ from the origin and its direction $\arg u$. Every other dot has its own p and $\arg u$, read off in the same way, and the circuit it stands for implements $R_z(\alpha)$ with $\alpha = -2\arg u$. The green dots lie between the dashed circle $p = p_{\min} = 0.9$ and the solid circle $p = 1$ and enter the library. The gray dots have $p < p_{\min}$ and are left out. The black squares have $v = 0$, so they succeed with certainty and implement the multiples of $\pi/4$.

Figure 10 shows these circuits for $k \leq 4$. For example, $k = 3$, $u = 1 - 2\omega^3$ and $v = \sqrt{2} - 1 = \omega - \omega^3 - 1$ give $\alpha = -2\arg(1 - 2\omega^3) \approx 0.337\pi$ and $p = (5 + 2\sqrt{2})/8 \approx 0.979$, with $T_V = 4$. Since $\alpha_f = \pi$, a failure applies Z, which costs no T to undo, and the circuit is repeated, at an expected $T_V/p \approx 4.09$ T. The threshold $p_{\min}$ trades the number of angles against their cost. A lower $p_{\min}$ admits more values of u, and hence more angles at each k, but by Eq. (A5) a fallback circuit then costs up to $(1 - p_{\min})\, T_{\text{fb}}$ more than T_V. Section II A gives the values of k, $p_{\min}$ and T_{fb} that we use.

Each law above has the form $c \log_2(1/\epsilon) + b$, fitted to random angles. In a circuit with R rotations and a total infidelity budget ϵ_{tot}, each rotation must be assigned an operator-norm error between $\sqrt{\epsilon_{\text{tot}}}/R$ and $\sqrt{\epsilon_{\text{tot}}/R}$, depending on whether the errors add coherently or incoherently. By these fits, the T count of the circuit is then at least

$$T_{\text{circuit}} \gtrsim R\Big[\tfrac{c}{2}\log_2\big(R/\epsilon_{\text{tot}}\big) + b\Big] \tag{A7}$$

This cost is linear in R. Improvements in synthesis have lowered c, but in every fit a generic rotation costs more than b. This is the rotation synthesis barrier. It applies to generic angles and is not a lower bound for particular ones. The example above implements a non-Clifford angle exactly at 4.09 T, below b for every law that uses an ancilla. SNAP does not attempt to lower c or b. Instead it moves most of the R rotations onto such angles and lets the remaining rotations absorb the resulting displacement, so that Eq. (A7) applies only to the few rotations that cannot be moved. Figure 8 sketches the two approaches.

Appendix B: Other moved stuff

1. Circuits and error

The state preparation circuits of Sec. III are built from two qubit orthogonal gates $G \in O(4)$ placed on a fixed layout, and Fig. 1(a) shows that each gate is written as six R_z rotations and Cliffords. With $s = (1 - \det G)/2 \in \{0, 1\}$, the product $\mathrm{CZ}^s G$ has unit determinant. Every real orthogonal matrix of unit determinant is a product of two $SU(2)$ matrices in the Bell basis [78], so that

$$G = \mathrm{CZ}^s\, W^\dagger \big[A_1 \otimes A_2\big] W \tag{B1}$$

where $A_1, A_2 \in SU(2)$ and W is the Clifford of Fig. 1(a) that maps the computational basis to the Bell basis,

$$\begin{aligned} W &= \mathrm{CNOT}_{1\to2}\,(H \otimes I)\,\mathrm{CNOT}_{2\to1}\,(S \otimes I) \\ &= \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 0 & i & 0 \\ 0 & 1 & 0 & i \\ 0 & -1 & 0 & i \\ 1 & 0 & -i & 0 \end{pmatrix} \end{aligned} \tag{B2}$$

Here $\mathrm{CNOT}_{a\to b}$ has control a and target b, and the matrix is written in the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ with qubit 1 on the left. Each $SU(2)$ factor is written in Z-Y-Z Euler form [79],

$$A_i = R_z(\alpha_i)\, R_y(\beta_i)\, R_z(\gamma_i) \tag{B3}$$

The middle rotation is an R_z rotation between Cliffords, since $R_y(\beta) = (SH)\, R_z(\beta)\, (SH)^\dagger$. Each gate therefore carries six R_z rotations, with angles $\alpha_1, \beta_1, \gamma_1, \alpha_2, \beta_2, \gamma_2$, and the decomposition of Eqs. (B1) to (B3) is exact. A circuit of M gates contains $R = 6M$ such rotations.

We number them $k = 1, \ldots, R$ in circuit order and write θ_k for the angle of rotation k, so that the vector $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_R) \in \mathbb{R}^R$ lists the six angles of every gate. The T count is carried by these R_z rotations alone, since Cliffords cost no T.

For the ground states of Sec. III A the gates are placed on a tree of long-range gates followed by nearest-neighbor layers, and the target $|\phi\rangle$ is a matrix product state (MPS) [75] of maximum bond dimension χ. The circuit prepares $|\psi(\boldsymbol{\theta})\rangle = C(\boldsymbol{\theta})\,|0\rangle^{\otimes n}$ on n qubits, as drawn in Fig. 11(b), and the error is the infidelity

$$\varepsilon(\boldsymbol{\theta}) = 1 - \left|\langle\phi|\psi(\boldsymbol{\theta})\rangle\right|^2 \tag{B4}$$

All infidelities in this paper follow Eq. (B4). Before any angle is snapped, we optimize all R angles as continuous variables to minimize $\varepsilon(\boldsymbol{\theta})$, using the continuous compile of Sec. B 3, which opens the loop of Fig. 11(a). We call the resulting angles the continuous optimum $\boldsymbol{\theta}^\star$. A circuit with a fixed layout generally cannot reach the target exactly, so $\boldsymbol{\theta}^\star$ leaves a residual infidelity $\varepsilon_0 = \varepsilon(\boldsymbol{\theta}^\star)$. The compiled circuit must satisfy $\varepsilon \le \varepsilon_{\text{target}}$ after all snaps, so ε_0 is part of the total budget, and the snaps may add at most $\varepsilon_{\text{target}} - \varepsilon_0$.

The time-evolution circuits of Sec. IV differ in both the circuit and the target. They act on an open chain of $N = 100$ qubits and apply the same parametrized two-qubit gate to every nearest-neighbor pair of one parity, in alternating brickwork layers, with a single-qubit gate on each end site of the odd layers. The angles of the bond gate are already R_z rotations up to Cliffords, so it needs no decomposition. The target is the exact evolution of a set of random product inputs, held as matrix product states, and the cost is the infidelity per site of Eq. (14), built from the corresponding overlaps. The method uses the cost only through these overlaps, so it carries over unchanged, apart from the environment calculation of Sec. B 2, which must account for angles shared by several rotations.

2. Environments

Each R_z rotation of a state preparation circuit appears exactly once, so the overlap is linear in it. Holding every angle other than θ_k fixed and contracting the rest of the tensor network [80], as shown in Fig. 11(b), leaves a 2×2 matrix E_k with

$$\begin{aligned}\langle\phi|\psi(\boldsymbol{\theta})\rangle &= \mathrm{Tr}\big[E_k\, R_z(\theta_k)\big] \\ &= (E_k)_{00}\, e^{-i\theta_k/2} + (E_k)_{11}\, e^{i\theta_k/2} \\ &= |(E_k)_{00}|\, e^{i(\eta_{k,0}-\theta_k/2)} + |(E_k)_{11}|\, e^{i(\eta_{k,1}+\theta_k/2)}\end{aligned} \tag{B5}$$

Here $\eta_{k,0}$ and $\eta_{k,1}$ are the phases of the diagonal entries, $(E_k)_{jj} = |(E_k)_{jj}|\, e^{i\eta_{k,j}}$ for $j = 0, 1$. Only the diagonal entries of E_k enter, since $R_z(\theta_k)$ in Eq. (1) is diagonal. Substituting Eq. (B5) into Eq. (B4) gives the infidelity as a function of θ_k alone,

$$\varepsilon(\theta_k) = 1 - a_k - b_k \cos(\theta_k - \varphi_k) \tag{B6}$$

where $a_k = |(E_k)_{00}|^2 + |(E_k)_{11}|^2$, $b_k = 2\,|(E_k)_{00}|\,|(E_k)_{11}|$, and $\varphi_k = \arg\left[(E_k)_{00}\,(E_k)_{11}^*\right] = \eta_{k,0} - \eta_{k,1}$ is the relative phase of the two diagonal entries. These three coefficients depend on every angle except θ_k.

The infidelity is therefore a sinusoid in each angle [81, 82], and a single contraction fixes it exactly at every value of that angle. A snap sets θ_k to an angle α of the library of Sec. II A, and we define the plug-in infidelity $\varepsilon_{\text{init}}(k, \alpha)$ as Eq. (B6) evaluated at $\theta_k = \alpha$. It is the infidelity of the circuit when rotation k is moved to α and every other angle keeps its current value. The free angles are then re-optimized starting from this point, and the re-optimization does not raise the infidelity, so $\varepsilon_{\text{init}}(k, \alpha)$ is an upper bound on the infidelity after the snap. It increases with the distance between α and φ_k, taken modulo 2π, so among the library angles of a given cost the best candidate is the one closest to φ_k.

The environments enter the loop of Fig. 11(a) directly after the continuous compile, and they are recomputed at the start of every round, since each accepted snap changes the environments of all other rotations. They supply the Hessian ranking of Sec. II B with the size of the rise, $\varepsilon_{\text{init}}(k, \alpha) - \varepsilon_{\text{cur}}$, where ε_{cur} is the infidelity at the current angles. The Hessian then determines only the fraction of this rise that survives once the free angles compensate.

The rise could also be estimated from the Hessian alone. At the continuous optimum, and after every re-optimization, each free angle sits at the minimum of its own sinusoid, $\theta_k = \varphi_k$, where the diagonal Hessian entry is $H_{kk} = \partial^2\varepsilon/\partial\theta_k^2 = b_k$. Moving the angle by δ then gives

$$\begin{aligned}\varepsilon_{\text{init}}(k, \varphi_k + \delta) - \varepsilon_{\text{cur}} &= b_k(1 - \cos\delta) \\ &= \frac{H_{kk}\,\delta^2}{2}\Big[1 - \frac{\delta^2}{12} + O(\delta^4)\Big]\end{aligned} \tag{B7}$$

For the small displacements of most library snaps the bracket equals one to high accuracy, and the quadratic estimate from the Hessian would suffice. The environment is still preferred, for three reasons. First, it is exact at every δ, including moves to Clifford angles, where δ reaches $\pi/4$ and the bracket in Eq. (B7) is 0.95, and it does not assume that θ_k sits exactly at its minimum, which holds only to the tolerance of the optimizer. Second, $\varepsilon_{\text{init}}$ is an upper bound at any point, so a snap with $\varepsilon_{\text{init}} \le \varepsilon_{\text{target}}$ is accepted without a trial. The first step of the loop of Fig. 11(a), described in Sec. II B, uses this to fix Clifford angles. Third, it is cheap. Evaluating the infidelity directly for every pair of rotation and library angle would take 121 920 R overlaps, whereas the environments need one contraction per rotation and one cosine per pair.

For an MPS target [75], we obtain these matrices from the environments of whole gates, as shown in Fig. 12. We label the gates $G_1, \ldots, G_M$ in circuit order and write $|\psi_{<j}\rangle = G_{j-1}\cdots G_1|0\rangle^{\otimes n}$ for the state prepared before gate j and $\langle\phi_{>j}| = \langle\phi|\, G_M \cdots G_{j+1}$ for the target pulled back through the gates after it, as in Fig. 12(a). The environment of gate j, drawn in Fig. 12(b), is the 4×4

matrix

$$E^{(4)} = \mathrm{Tr}_{\bar{j}}\big[\,|\psi_{<j}\rangle\langle\phi_{>j}|\,\big] \qquad \langle\phi|\psi\rangle = \mathrm{Tr}\big[E^{(4)}G_j\big] \tag{B8}$$

where $\mathrm{Tr}_{\bar{j}}$ traces out every qubit on which G_j does not act, and reconnecting G_j as in Fig. 12(c) returns the overlap. Caching $|\psi_{<j}\rangle$ and $\langle\phi_{>j}|$ as MPS from both ends of the circuit gives the environments of all R rotations at a cost of $O(nR\chi^3)$ per round. To reach a single rotation, let rotation k act on the first qubit of gate j and split the gate as in Fig. 12(d), $G_j = X_a\,[R_z(\theta_k)\otimes I]\,X_b$, where X_b and X_a collect the factors of Eqs. (B1) to (B3) applied before and after the rotation. Then

$$E_k = \mathrm{Tr}_2\big[X_b\,E^{(4)}X_a\big] \tag{B9}$$

where Tr_2 traces out the second qubit of the gate, and the two qubits exchange roles for a rotation on the second qubit. Equation (B9) needs only products of 4×4 matrices, at a cost independent of n and χ. The same gate environments drive the continuous compile of Sec. B 3, following Ref. [80].

In the time-evolution circuits of Sec. IV the same gate acts on every bond of a sublayer, so an angle θ is shared by the $m = N/2$ rotations of an even sublayer, or the $N/2-1$ of an odd one, and the overlap is no longer linear in it. For each input state, with $|\phi\rangle$ its exact evolution,

$$\langle\phi|\psi(\theta)\rangle = \sum_{l=0}^{m} c_l\, e^{i(2l-m)\theta/2} \tag{B10}$$

$$\big|\langle\phi|\psi(\theta)\rangle\big|^2 = \sum_{q=-m}^{m} d_q\, e^{iq\theta} \tag{B11}$$

where the c_l do not depend on θ and $d_q = \sum_{l-l'=q} c_l\, c_{l'}^*$. This form holds for the angles a, b, c, e and f of Eq. (13). The staggered angle d enters each bond as $R_z(d)\otimes R_z(-d)$, whose phases are e^0 and $e^{\pm i\theta}$ rather than $e^{\pm i\theta/2}$, so its overlap has $2m+1$ terms instead of $m+1$. On the open chain we hold the state before sublayer ℓ, $|a_\ell\rangle$, and the target pulled back through the later sublayers, $\langle b_\ell|$, as matrix product states, and we contract $\langle b_\ell|L_\ell(\theta)|a_\ell\rangle$ without truncation, so it is exactly the trigonometric polynomial described above. Its $m+1$ coefficients c_l, or $2m+1$ for the staggered angle, follow from a discrete Fourier transform of the overlap at as many equally spaced values of θ [83], 51 or 101 at $N = 100$, and one more value at the current angle checks the fit. Dividing by the norms of $|a_\ell\rangle$ and $\langle b_\ell|$ then gives $\varepsilon_{\mathrm{init}}$ exactly at every library angle.

3. Continuous compile

Snapping starts from the continuous optimum $\boldsymbol{\theta}^\star$ in Sec. B 1. For the state preparation circuits we compute it gate by gate, following Evenbly and Vidal [80]. With every other gate held fixed, the overlap is linear in the gate, $\langle\phi|\psi\rangle = \mathrm{Tr}[E^{(4)}G]$, where $E^{(4)}$ is the gate environment of Eq. (B8) and Fig. 12(b). Let $\omega = \arg\langle\phi|\psi\rangle$ be the phase of the current overlap. Since every candidate gate G' is real,

$$\big|\mathrm{Tr}[E^{(4)}G']\big| \ge \mathrm{Re}\big(e^{-i\omega}\,\mathrm{Tr}[E^{(4)}G']\big) = \mathrm{Tr}[KG'] \qquad K = \mathrm{Re}\big[e^{-i\omega}E^{(4)}\big] \tag{B12}$$

with equality at the current gate. Over the real orthogonal matrices, $\mathrm{Tr}[KG']$ is maximized by

$$G' = VU^T, \qquad K = U\Sigma V^T \tag{B13}$$

where $U\Sigma V^T$ is the singular value decomposition of K, and its maximum is $\mathrm{Tr}\,\Sigma$. By Eq. (B12), replacing the gate by Eq. (B13) never lowers $|\langle\phi|\psi\rangle|$. Since ω changes with the gate, we recompute it and repeat the update a few times per gate. The maximum runs over all of $O(4)$ and does not fix $\det G$, so a gate can change branch during the compile, and the factor CZ^s of Eq. (B1) records the branch on which it ends.

A sweep applies this update to every gate in circuit order. Each update uses the environment of Eq. (B8), with $|\psi_{<j}\rangle$ and $\langle\phi_{>j}|$ held as MPS of maximum bond dimension χ, so a sweep costs about as much as one computation of all environments. We repeat sweeps until $|\langle\phi|\psi\rangle|$ stops increasing. The gates start as random orthogonal matrices, and a larger layout starts from the optimum of the smaller layout it contains, with each added gate set to the identity.

The compile returns one orthogonal matrix per gate, which we convert to angles through Eqs. (B1) to (B3) and the circuit of Fig. 1(a). The sign s follows from $\det G$, the two factors from $A_1\otimes A_2 = W\,\mathrm{CZ}^s G\,W^\dagger$, and the six angles from the Z-Y-Z Euler form of each factor. The result is $\boldsymbol{\theta}^\star$, and its infidelity is ε_0.

Once snapping begins, a gate with a snapped angle can no longer be updated as a whole, so we re-optimize the free angles one at a time. Each free angle is set to the minimum of its sinusoid, Eq. (B6),

$$\theta_k \leftarrow \varphi_k \tag{B14}$$

with E_k recomputed from Eq. (B9) along the sweep, as in Fig. 12(d). This is the coordinate update of Refs. [81, 82], and it never raises the infidelity. We repeat it until $|\langle\phi|\psi\rangle|$ stops increasing or a maximum number of sweeps is reached.

The time-evolution circuits of Sec. IV share each angle among many gates, so no gate can be updated on its own. We minimize their cost directly over the bulk and end angles, on the open chain, by trust-region Newton steps. The gradient comes from the adjoint method on the matrix product states. The forward pass stores the state before every sublayer, a backward pass carries the target back through the circuit one sublayer at a time, and contracting the two around each gate gives the derivative of every overlap with respect to that gate. The

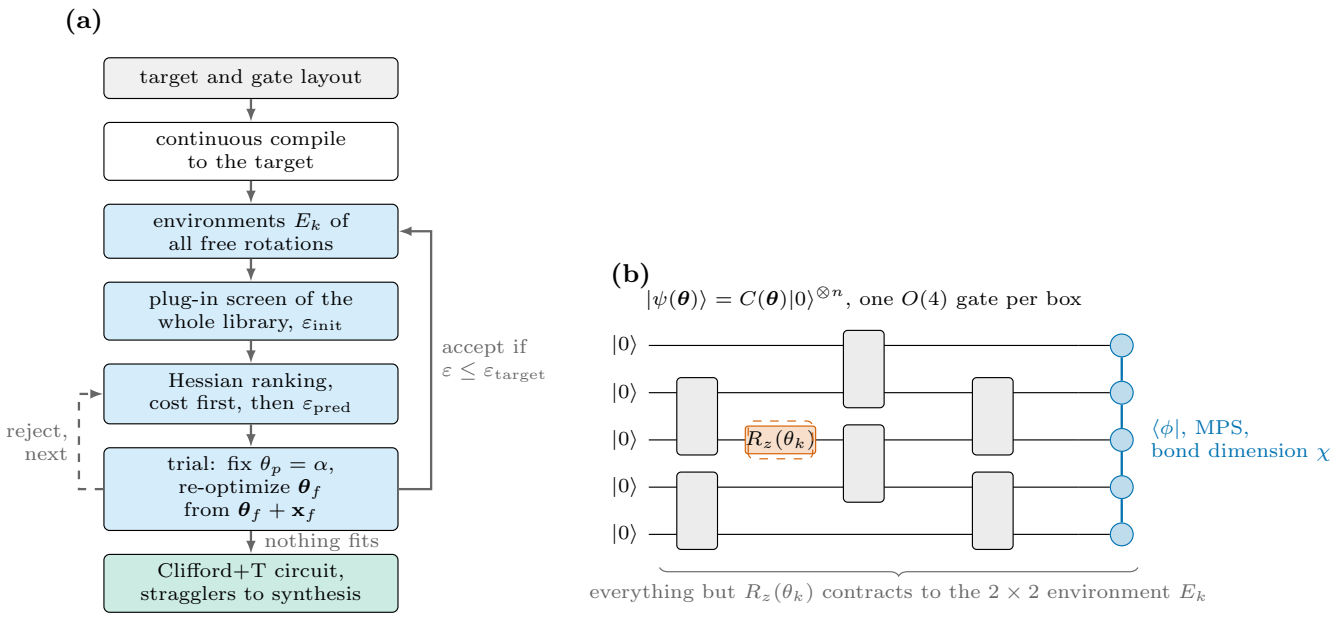


FIG. 11. (a) The SNAP loop. The blue boxes repeat once per accepted snap, and a trial is accepted when the re-optimized infidelity is within the budget and the budget rules of Sec. II B are met. (b) The environment of one rotation, shown for an MPS target. Removing $R_z(\theta_k)$ from the overlap $\langle\phi|\psi(\boldsymbol{\theta})\rangle$ and contracting the rest of the network leaves a 2×2 matrix E_k, from which the exact infidelity at every value of θ_k follows in closed form, as derived in Sec. B 2. Figure 12 shows how E_k follows from the environment of its gate. lets split up this figure, move fig. a to Snap compilation section

Hessian is that of Eq. (B16), and each one serves several Newton steps before it is recomputed. At each depth we run three optimizations, started from the Trotter angles of that depth, from the optimum at the previous depth padded with identity layers, and from the Trotter angles with 0.1 rad of noise, and we keep the best of the three. Each runs until the error falls by less than a part in 10^6 in a step or reaches the truncation noise, with no cap on the number of iterations. Before the first snap, further Newton steps continue until the predicted decrease of a step is below 10^{-6} of the budget. The same method, with the Hessian recomputed every 20 steps, re-optimizes the free angles after each snap, in the trial step of Fig. 11(a).

4. Hessian

projected onto the positive semidefinite matrices by setting its negative eigenvalues to zero. At a minimum H is already positive semidefinite and the projection leaves it unchanged, so its diagonal entries are the curvatures H_{kk} of Eq. (B7).

For the state preparation circuits we compute H from the overlap $o(\boldsymbol{\theta}) = \langle\phi|\psi(\boldsymbol{\theta})\rangle$. Differentiating Eq. (B4) twice gives

$$H_{kl} = -2\,\mathrm{Re}\big[(\partial_k o)^*\,\partial_l o + o^*\,\partial_k\partial_l o\big] \tag{B15}$$

where $\partial_k = \partial/\partial\theta_k$. For rotation k in gate j, the gate environment of Eq. (B8) gives $\partial_k o = \mathrm{Tr}[E^{(4)}\partial_k G_j]$, and the same environment gives $\partial_k\partial_l o$ when rotation l is in the same gate. When rotation l is in a later gate j', we propagate $|\partial_k\psi_{<j'}\rangle = G_{j'-1}\cdots G_{j+1}\,(\partial_k G_j)\,|\psi_{<j}\rangle$ as an MPS [75] and use it in place of $|\psi_{<j'}\rangle$ in the environment of gate j'. For the time-evolution circuits, H is the Hessian of the training error of Eq. (14) in place of ε, over the free bulk and end angles. We take central differences of its exact adjoint gradient $\mathbf{g}$,

$$H_{kl} = \frac{g_l(\boldsymbol{\theta} + h\,\mathbf{e}_k) - g_l(\boldsymbol{\theta} - h\,\mathbf{e}_k)}{2h} \tag{B16}$$

where $\mathbf{e}_k$ is the unit vector along θ_k and $h = 10^{-4}$, and symmetrize the result. Only the rows of free angles are computed, and the $2|\mathcal{F}|$ gradients of one Hessian run in parallel over the training inputs. The finite-difference Hessian is noisy at about 5×10^{-8} of its largest entry, so the pseudo-inverse in Eq. (8) discards singular values below 10^{-8} of the largest.

5. Snap protocol

The loop of Fig. 11(a) repeats four steps. First, we compute the environments of the free rotations and snap zero-cost rotations directly. Among all free rotations and all Clifford angles, we take the pair with the lowest $\varepsilon_{\text{init}}$, and if $\varepsilon_{\text{init}} \le \varepsilon_{\text{target}}$ we snap it, re-optimize the free angles and repeat. These snaps need no trial, since $\varepsilon_{\text{init}}$ is an upper bound. Second, we compute H, and for each free rotation p and each distinct cost c in the library,

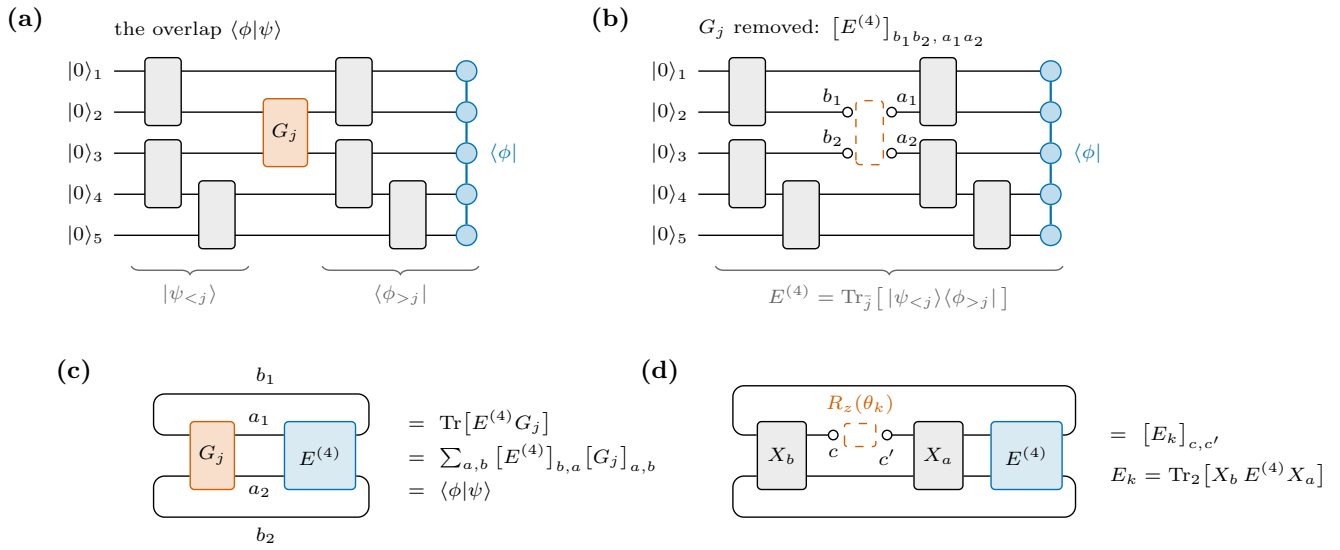


FIG. 12. Gate and rotation environments. (a) The overlap $\langle\phi|\psi\rangle$ for a layout of gates $G_1, \ldots, G_M$, with gate G_j highlighted. The gates before G_j prepare $|\psi_{<j}\rangle$, and the gates after it, applied in reverse to the target, give $\langle\phi_{>j}|$. (b) Removing G_j leaves four open legs, b_1 and b_2 on the input side and a_1 and a_2 on the output side, and contracting the rest of the network gives the 4×4 environment of Eq. (B8). (c) Reconnecting G_j closes the trace and returns the overlap. (d) Inside the gate, $G_j = X_a[R_z(\theta_k) \otimes I]X_b$. Removing $R_z(\theta_k)$ and closing the other qubit gives the 2×2 environment E_k of Eq. (B9), with open legs c and c'.

including $c = 0$, we keep the angle of cost c with the lowest $\varepsilon_{\mathrm{init}}(p, \alpha)$, which by Sec. B 2 is the one closest to φ_p. We discard every candidate with $\varepsilon_{\mathrm{pred}} > \varepsilon_{\mathrm{target}}$. Third, we order the remaining candidates by cost, and within one cost by $\varepsilon_{\mathrm{pred}}$, with ties broken by the largest component of $\mathbf{x}_f$, so that the snaps that disturb the free angles least come first. In the time-evolution circuits an angle is shared by m rotations, and the cost of a candidate is $m\,c(\alpha)$. Fourth, we try a fixed number of candidates at each cost, in this order. A trial fixes $\theta_p = \alpha$, starts the free angles at $\boldsymbol{\theta}_f + \mathbf{x}_f$, and re-optimizes them with the method of Sec. B 3, which is the coordinate update of Eq. (B14) [81, 82] for the state preparation circuits and trust-region Newton steps for the time-evolution circuits. We accept the snap if the re-optimized infidelity is at most $\varepsilon_{\mathrm{target}}$ and return to the first step. The quadratic model only orders the trials, and the re-optimization alone decides acceptance. When no candidate at any cost is accepted, the loop ends and the rotations still free are synthesized as described in Sec. II A. Including $c = 0$ in the second step lets the loop try a Clifford angle that fails the plug-in test of the first step but is predicted to fit once the free angles respond. Sections III A and IV A give the settings of the loop for each application.

The state preparation runs of Sec. III use the loop as described. In the time-evolution runs of Sec. IV the budget is often only a few times ε_0, and there the loop can spend the whole budget on snaps, leaving the rotations it never reaches to be synthesized at whatever precision remains, which costs more than the last snaps saved. We add four rules for these runs. First, a snap is accepted only if the budget it leaves suffices to synthesize every other free rotation at no more than the cost of the most expensive library angle. We estimate the budget that each such rotation needs from its curvature H_{kk}, and we cap the total at a fixed fraction of $\varepsilon_{\mathrm{target}}$, scaled by the fraction of angles still free, so that it cannot exceed the budget at tight targets. Second, the prediction of Eq. (11) is optimistic when $\varepsilon_{\mathrm{target}}$ is only a few times ε_0, so we limit how much of the remaining budget one snap may consume,

$$\varepsilon_{\mathrm{after}} - \varepsilon_{\mathrm{cur}} \le \lambda\,\kappa\,\frac{\varepsilon_{\mathrm{target}} - \varepsilon_{\mathrm{cur}}}{|\mathcal{F}|} \tag{B17}$$

where $\varepsilon_{\mathrm{after}}$ is the re-optimized infidelity of the trial and $|\mathcal{F}|$ is the number of free rotations, so that $(\varepsilon_{\mathrm{target}} - \varepsilon_{\mathrm{cur}})/|\mathcal{F}|$ is the fair share of the remaining budget. The constant κ is the number of fair shares that one snap may take, and $\lambda \ge 1$ relaxes the bound. We apply the same bound to $\varepsilon_{\mathrm{pred}}$ when forming the candidates. Third, Eq. (B17) is a preference and not a veto. Each run starts at $\lambda = 1$. If a round accepts no candidate, we repeat it with a larger λ, up to removing the bound, and later rounds keep the value of λ that succeeded. A round also ends after a fixed number of trials. Fourth, every prefix of the accepted sequence is itself a valid compilation. Let ε_j be the infidelity after the first j of the S accepted snaps, $\mathcal{F}_j$ the set of rotations still free at that point, and $T_{\mathrm{syn}}(\mathcal{F}_j, \varepsilon_{\mathrm{target}} - \varepsilon_j)$ the expected T count of synthesizing them within the remaining budget at the fallback cost of

Sec. II A. We report

$$T = \min_{0\le j\le S} \Big[\sum_{i=1}^{j} m_i\, c_i + T_{\rm syn}\big(\mathcal{F}_j,\, \varepsilon_{\rm target} - \varepsilon_j\big) \Big] \quad \text{(B18)}$$

where c_i is the library cost of the ith accepted snap and m_i is the number of rotations that share its angle. On the open chain $T_{\rm syn}$ also covers the end angles, each weighted by $2/N$ against a bulk angle, since an end gate appears once per circuit and a bulk gate $N/2$ times.

The loop can accept a snap whose plug-in infidelity exceeds the entire remaining budget, as for the Clifford angle in Fig. 2(b), because by Eq. (11) only the fraction r_p of the rise survives. A compiler that fixes every rotation at once cannot accept such a snap, however it allocates the budget.

Appendix C: Time-evolution details

1. Training

Every circuit is trained on the open chain of $N = 100$ sites. The training set is 12 random product inputs. Their references are a fourth-order product formula with step 0.1, truncated at a discarded weight of 10^{-14} and a bond dimension of at most 2048, and the circuit itself is truncated at 10^{-14} and 1024. The parameters are the bulk angles, one set per sublayer, and the six $R_zR_yR_z$ angles of the two end gates on every odd sublayer, which start from the single-qubit factor of the missing bond gate. Each start is optimized as in Sec. B 3. The ladder adds depths from a fixed grid and stops once the floor falls by less than a factor of 1.5 over eight sublayers, or reaches 10^{-10}. Each depth also records its error on the 8 held-out inputs of the evaluation.

2. Compiling

The optimizer, the Newton steps before the first snap and the re-optimization of each trial follow Sec. B 3, and the Hessian follows Eq. (B16). While compiling, the library is priced with a fallback cost of $T_{\rm fb} = 22$ for the correction of a failed library circuit. The budget rules of the loop are those of the snap protocol appendix, Eqs. (B17) and (B18). The end angles are in the free block of every Hessian, so the predicted response moves them too, and they enter the stage that snaps zero-cost angles directly. They are not candidates of the Hessian rounds. When the loop stops, each end angle is weighted by $2/N$ against a bulk angle, since an end gate appears once and a bulk gate $N/2$ times, and receives a library angle or a synthesis budget by the same allocation as the bulk stragglers. The end gates are then simplified on the chain within the budget: gates near the identity are dropped, gates whose middle angle is near zero become single R_z rotations, and angles within 10^{-2} of a multiple of $\pi/4$ are set to it. After compiling we reprice the library on $T_{\rm fb} \in \{18, 20, \ldots, 30\}$ and keep the smallest value whose total error is within 1% of the total at $T_{\rm fb} = 30$.

3. Matrix product states

The evaluation evolves both $U|\phi_k\rangle$ and $e^{-iHt}|\phi_k\rangle$ as matrix product states on the open chain. After each two-qubit gate we keep the fewest singular values whose discarded weight is at most 10^{-12}, up to a bond dimension of 1024, and 10^{-14} for the budgets 3×10^{-9} and 10^{-9}. The reference is a fourth-order product formula with step 0.1 at $h_x = 1.6$ and 0.2 at $h_x = 0$, where its error is about 2×10^{-13} per site. For each field and time we repeat one input at a cutoff 100 times smaller, and we flag a point if its infidelity changes by more than 5%.

4. Fallback branches

Each synthesized rotation i takes its fallback branch with probability $q_i \le 0.1$ per use, and its fallback differs from its success branch by δ_i in the diamond norm. In the $N = 100$ simulation we draw the branch of every synthesized rotation at random, bulk and end gates alike, with 4 draws per input for $t \le 4$, and average. A library circuit whose failure is not a multiple of $\pi/4$ is corrected by a synthesized rotation whose circuit the library does not carry, so for these we use a bound. With $\Lambda = \sum_i q_i\delta_i$ and $Q_2 = \frac{1}{2}\sum_i q_i\delta_i^2$ over all such instances,

$$1 - F \le \Big(\sqrt{1-F_s} + \frac{\Lambda}{2}\Big)^2 + Q_2, \quad \text{(C1)}$$

where F_s is the fidelity with every library circuit on its success branch. The bound follows because $\sqrt{1-F}$ is a metric on pure states and each fallback moves the output by at most $\delta_i/2$ in it. Its contribution is small, since these corrections are synthesized to about 10^{-5}. A point that has not been drawn yet carries the same bound over all its synthesized rotations, which can only overstate its error.

5. Baselines

The measured baseline scans product formulas of order 1, 2 and 4 over the number of steps r and the synthesis precision $\epsilon_{\rm syn} \in \{10^{-1}, 3\times10^{-2}, \ldots, 10^{-7}\}$. The product formulas are not trained. We screen them by their error on a 10-site ring and evaluate the cheapest at each error at $N = 100$ on the open chain, exactly as for SNAP. At $h_x = 1.6$ the error of the step-0.2 reference partly cancels that of a product formula, so every point with $\epsilon \le 10^{-6}$ there is evaluated against step-0.1 references. The rigorous baseline converts the operator-norm bound δ of Ref. [76], for the Trotter error and the synthesis together, into a guarantee on the error per site through

$1 - F \leq \delta^2$, which gives $\epsilon \leq 1 - (1 - \delta^2)^{1/N}$. Each of the R rotations receives a synthesis precision δ_{syn}/R, where δ_{syn} is the part of δ left after the Trotter error.

ACKNOWLEDGMENT

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, as part of the Genesis Mission National Science and Technology Challenges under Award Number(s) [Enter the award number(s)].

[1] R. P. Feynman et al., Simulating physics with computers, Feynman and computation **22**, 133 (2018).

[2] S. Lloyd, Universal quantum simulators, science **273**, 1073 (1996).

[3] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. S. L. Brandao, D. A. Buell, et al., Quantum supremacy using a programmable superconducting processor, Nature **574**, 505 (2019), arXiv:1910.11333.

[4] X. Mi, P. Roushan, C. Quintana, S. Mandrà, J. Marshall, C. Neill, F. Arute, K. Arya, J. Atalaya, R. Babbush, et al., Information scrambling in quantum circuits, Science **374**, 1479 (2021), arXiv:2101.08870.

[5] X. Mi, M. Ippoliti, C. Quintana, A. Greene, Z. Chen, J. Gross, F. Arute, K. Arya, J. Atalaya, R. Babbush, et al., Time-crystalline eigenstate order on a quantum processor, Nature **601**, 531 (2022), arXiv:2107.13571.

[6] Y. Kim, A. Eddins, S. Anand, K. X. Wei, E. van den Berg, S. Rosenblatt, H. Nayfeh, Y. Wu, M. Zaletel, K. Temme, and A. Kandala, Evidence for the utility of quantum computing before fault tolerance, Nature **618**, 500 (2023).

[7] T. I. Andersen, N. Astrakhantsev, A. H. Karamlou, J. Berndtsson, J. Motruk, A. Szasz, J. A. Gross, A. Schuckert, T. Westerhout, Y. Zhang, et al., Thermalization and criticality on an analogue-digital quantum simulator, Nature **638**, 79 (2025), arXiv:2405.17385.

[8] Google Quantum AI and Collaborators, Observation of constructive interference at the edge of quantum ergodicity, Nature **646**, 825 (2025), arXiv:2506.10191.

[9] R. Haghshenas, E. Chertkov, M. Mills, W. Kadow, S.-H. Lin, Y. H. Chen, C. Cade, I. Niesen, T. Begušić, M. S. Rudolph, et al., Digital quantum magnetism on a trapped-ion quantum computer, Nature **653**, 56 (2026), arXiv:2503.20870.

[10] P. W. Shor, Scheme for reducing decoherence in quantum computer memory, Phys. Rev. A **52**, R2493(R) (1995).

[11] B. Eastin and E. Knill, Restrictions on transversal encoded quantum gate sets, Physical Review Letters **102**, 110502 (2009), arXiv:0811.4262.

[12] A. Y. Kitaev, Quantum computations: algorithms and error correction, Russian Mathematical Surveys **52**, 1191 (1997).

[13] C. M. Dawson and M. A. Nielsen, The Solovay-Kitaev algorithm, Quantum Information and Computation **6**, 81 (2006), arXiv:quant-ph/0505030.

[14] N. J. Ross and P. Selinger, Optimal ancilla-free Clifford+T approximation of z-rotations, Quantum Information and Computation **16**, 901 (2016), arXiv:1403.2975.

[15] V. Kliuchnikov, K. Lauter, R. Minko, A. Paetznick, and C. Petit, Shorter quantum circuits via single-qubit gate approximation, Quantum **7**, 1208 (2023), arXiv:2203.10064; non-mixed fallback cost $1.03 \log_2(1/\varepsilon) + 5.75$.

[16] M. Reiher, N. Wiebe, K. M. Svore, D. Wecker, and M. Troyer, Elucidating reaction mechanisms on quantum computers, Proceedings of the National Academy of Sciences **114**, 7555 (2017), arXiv:1605.03590.

[17] A. M. Childs, D. Maslov, Y. Nam, N. J. Ross, and Y. Su, Toward the first quantum simulation with quantum speedup, Proceedings of the National Academy of Sciences **115**, 9456 (2018), arXiv:1711.10980.

[18] I. D. Kivlichan, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, W. Sun, Z. Jiang, N. Rubin, A. Fowler, A. Aspuru-Guzik, H. Neven, and R. Babbush, Improved fault-tolerant quantum simulation of condensed-phase correlated electrons via Trotterization, Quantum **4**, 296 (2020), arXiv:1902.10673.

[19] Z. Sun and B. Koczor, Space and time cost of continuous rotations in surface codes, Quantum Sci. Technol. **11**, 025046 (2026), arXiv:2508.06236.

[20] H. F. Trotter, On the product of semi-groups of operators, Proceedings of the American Mathematical Society **10**, 545 (1959).

[21] M. Suzuki, General theory of fractal path integrals with applications to many-body theories and statistical physics, Journal of Mathematical Physics **32**, 400 (1991).

[22] D. S. Abrams and S. Lloyd, Simulation of many-body Fermi systems on a universal quantum computer, Physical Review Letters **79**, 2586 (1997), arXiv:quant-ph/9703054.

[23] A. M. Childs, Y. Su, M. C. Tran, N. Wiebe, and S. Zhu, Theory of Trotter error with commutator scaling, Physical Review X **11**, 011020 (2021), arXiv:1912.08854.

[24] E. T. Campbell, Early fault-tolerant simulations of the Hubbard model, Quantum Science and Technology **7**, 015007 (2022), arXiv:2012.09238.

[25] N. Yoshioka, T. Okubo, Y. Suzuki, Y. Koizumi, and W. Mizukami, Hunting for quantum-classical crossover in condensed matter problems, npj Quantum Information **10**, 45 (2024), arXiv:2210.14109.

[26] S. Bravyi and A. Kitaev, Universal quantum computation with ideal Clifford gates and noisy ancillas, Phys. Rev. A **71**, 022316 (2005), arXiv:quant-ph/0403025.

[27] E. Knill, Quantum computing with realistically noisy devices, Nature **434**, 39 (2005), arXiv:quant-ph/0410199.

[28] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Surface codes: Towards practical large-scale quantum computation, Physical Review A **86**, 032324 (2012), arXiv:1208.0928.

[29] S. Bravyi and J. Haah, Magic-state distillation with low overhead, Physical Review A **86**, 052329 (2012),

arXiv:1209.2426.
[30] E. T. Campbell, B. M. Terhal, and C. Vuillot, Roads towards fault-tolerant universal quantum computation, Nature **549**, 172 (2017), arXiv:1612.07330.
[31] D. Litinski, Magic state distillation: Not as costly as you think, Quantum **3**, 205 (2019), arXiv:1905.06903.
[32] C. Gidney and A. G. Fowler, Efficient magic state factories with a catalyzed $|ccz\rangle \to 2|t\rangle$ transformation, Quantum **3**, 135 (2019), arXiv:1812.01238.
[33] C. Gidney, N. Shutty, and C. Jones, Magic state cultivation: growing T states as cheap as CNOT gates (2024), arXiv:2409.17595.
[34] E. Rosenfeld, C. Gidney, G. Roberts, A. Morvan, N. Lacroix, D. Kafri, J. Marshall, M. Li, V. Sivak, D. Abanin, et al., Magic state cultivation on a superconducting quantum processor (2025), arXiv:2512.13908.
[35] L. Daguerre and I. H. Kim, Code switching revisited: Low-overhead magic state preparation using color codes, Physical Review Research **7**, 023080 (2025), arXiv:2410.07327.
[36] L. Daguerre, R. Blume-Kohout, N. C. Brown, D. Hayes, and I. H. Kim, Experimental demonstration of high-fidelity logical magic states from code switching, Physical Review X **15**, 041008 (2025), arXiv:2506.14169.
[37] S. Dasu, S. Burton, K. Mayer, D. Amaro, J. A. Gerber, K. Gilmore, D. Gresh, D. DelVento, A. C. Potter, and D. Hayes, Breaking even with magic: demonstration of a high-fidelity logical non-Clifford gate (2025), arXiv:2506.14688.
[38] C. Gidney, Halving the cost of quantum addition, Quantum **2**, 74 (2018), arXiv:1709.06648.
[39] C. Jones, Distillation protocols for Fourier states in quantum computing, Quantum Information and Computation **14**, 560 (2014), arXiv:1303.3066.
[40] Y. Nam, Y. Su, and D. Maslov, Approximate quantum Fourier transform with $O(n \log(n))$ T gates, npj Quantum Information **6**, 26 (2020), arXiv:1803.04933.
[41] E. Campbell, Shorter gate sequences for quantum computing by mixing unitaries, Physical Review A **95**, 042306 (2017), arXiv:1612.02689.
[42] M. B. Hastings, Turning gate synthesis errors into incoherent errors, Quantum Information and Computation **17**, 488 (2017), arXiv:1612.01011.
[43] B. Koczor, J. J. L. Morton, and S. C. Benjamin, Probabilistic interpolation of quantum rotation angles, Physical Review Letters **132**, 130602 (2024), arXiv:2305.19881.
[44] G. Duclos-Cianci and D. Poulin, Reducing the quantum-computing overhead with complex gate distillation, Physical Review A **91**, 042315 (2015), arXiv:1403.5280.
[45] A. J. Landahl and C. Cesare, Complex instruction set computing architecture for performing accurate quantum Z rotations with less magic (2013), arXiv:1302.3240.
[46] E. T. Campbell and J. O'Gorman, An efficient magic state approach to small angle rotations, Quantum Science and Technology **1**, 015007 (2016), arXiv:1603.04230.
[47] Y. Akahoshi, K. Maruyama, H. Oshima, S. Sato, and K. Fujii, Partially fault-tolerant quantum computing architecture with error-corrected Clifford gates and space-time efficient analog rotations, PRX Quantum **5**, 010337 (2024), arXiv:2303.13181.
[48] R. Toshio, Y. Akahoshi, J. Fujisaki, H. Oshima, S. Sato, and K. Fujii, Practical quantum advantage on partially fault-tolerant quantum computer, Physical Review X **15**, 021057 (2025), arXiv:2408.14848.
[49] M. Amy, D. Maslov, and M. Mosca, Polynomial-time T-depth optimization of Clifford+T circuits via matroid partitioning, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems **33**, 1476 (2014), arXiv:1303.2042.
[50] Y. Nam, N. J. Ross, Y. Su, A. M. Childs, and D. Maslov, Automated optimization of large quantum circuits with continuous parameters, npj Quantum Information **4**, 23 (2018), arXiv:1710.07345.
[51] L. E. Heyfron and E. T. Campbell, An efficient quantum compiler that reduces T count, Quantum Science and Technology **4**, 015004 (2019), arXiv:1712.01557.
[52] A. Kissinger and J. van de Wetering, Reducing the number of non-Clifford gates in quantum circuits, Physical Review A **102**, 022406 (2020), arXiv:1903.10477.
[53] M. Bothe, C. Sünderhauf, M. J. Witham, E. T. Campbell, and N. S. Blunt, More efficient Clifford+T synthesis for small-angle rotations and application to Trotterization (2026), arXiv:2605.31544.
[54] H. Morisaki, K. Sano, and S. Akibue, Optimal ancilla-free Clifford+T synthesis for general single-qubit unitaries (2025), arXiv:2510.05816.
[55] S.-H. Lin, R. Dilip, A. G. Green, A. Smith, and F. Pollmann, Real- and imaginary-time evolution with compressed quantum circuits, PRX Quantum **2**, 010342 (2021), arXiv:2008.10322.
[56] C. Mc Keever and M. Lubasch, Classically optimized hamiltonian simulation, Physical Review Research **5**, 023146 (2023), arXiv:2205.11427.
[57] A. Kotil, R. Banerjee, Q. Huang, and C. B. Mendl, Riemannian quantum circuit optimization for hamiltonian simulation, Journal of Physics A: Mathematical and Theoretical **57**, 135303 (2024), arXiv:2212.07556.
[58] L. Causer, F. Jung, A. Mitra, F. Pollmann, and A. Gammon-Smith, Scalable simulation of nonequilibrium quantum dynamics via classically optimized unitary circuits, Physical Review Research **6**, 033062 (2024), arXiv:2312.14245.
[59] C. Mc Keever and M. Lubasch, Towards adiabatic quantum computing using compressed quantum circuits, PRX Quantum **5**, 020362 (2024), arXiv:2311.05544.
[60] J. Gibbs and L. Cincio, Deep circuit compression for quantum dynamics via tensor networks, Quantum **9**, 1789 (2025), arXiv:2409.16361.
[61] I. N. M. Le, S. Sun, and C. B. Mendl, Riemannian quantum circuit optimization based on matrix product operators, Quantum **9**, 1833 (2025), arXiv:2501.08872.
[62] J. Gibbs and L. Cincio, Learning circuits with infinite tensor networks (2025), arXiv:2506.02105.
[63] Y. Zhang, R. Wiersema, J. Carrasquilla, L. Cincio, and Y. B. Kim, Scalable quantum dynamics compilation via quantum machine learning, Physical Review Research **8**, 023128 (2026), arXiv:2409.16346.
[64] A. G. Fowler, Constructing arbitrary Steane code single logical qubit fault-tolerant gates, Quantum Information and Computation **11**, 867 (2011), arXiv:quant-ph/0411206.
[65] A. Bocharov and K. M. Svore, Resource-optimal single-qubit quantum circuits, Physical Review Letters **109**, 190501 (2012), arXiv:1206.3223.
[66] P. Selinger, Efficient Clifford+T approximation of single-qubit operators, Quantum Information and Computation **15**, 159 (2015), arXiv:1212.6253.

[67] V. Kliuchnikov, D. Maslov, and M. Mosca, Asymptotically optimal approximation of single qubit unitaries by Clifford and T circuits using a constant number of ancillary qubits, Physical Review Letters **110**, 190502 (2013), arXiv:1212.0822.

[68] V. Kliuchnikov, D. Maslov, and M. Mosca, Fast and efficient exact synthesis of single-qubit unitaries generated by Clifford and T gates, Quantum Information and Computation **13**, 607 (2013), arXiv:1206.5236; exact T-count via the ring reduction.

[69] B. Giles and P. Selinger, Remarks on Matsumoto and Amano's normal form for single-qubit Clifford+T operators (2013), arXiv:1312.6584.

[70] A. Paetznick and K. M. Svore, Repeat-until-success: non-deterministic decomposition of single-qubit unitaries, Quantum Information and Computation **14**, 1277 (2014), arXiv:1311.1074.

[71] A. Bocharov, M. Roetteler, and K. M. Svore, Efficient synthesis of universal repeat-until-success quantum circuits, Phys. Rev. Lett. **114**, 080502 (2015), arXiv:1404.5320.

[72] A. Bocharov, M. Roetteler, and K. M. Svore, Efficient synthesis of probabilistic quantum circuits with fallback, Phys. Rev. A **91**, 052317 (2015), arXiv:1409.3552.

[73] S. Boyd and L. Vandenberghe, Convex Optimization (Cambridge University Press, Cambridge, 2004) appendix A.5.5.

[74] M. C. Caro, H.-Y. Huang, N. Ezzell, J. Gibbs, A. T. Sornborger, L. Cincio, P. J. Coles, and Z. Holmes, Out-of-distribution generalization for learning quantum dynamics, Nat. Commun. **14**, 3751 (2023), arXiv:2204.10268.

[75] U. Schollwöck, The density-matrix renormalization group in the age of matrix product states, Ann. Phys. **326**, 96 (2011), arXiv:1008.3477.

[76] Z. Liu, Z.-Y. Wei, M. Foss-Feig, A. V. Gorshkov, and A. M. Childs, Ordering-aware theory of Trotter error (2026), arXiv:2610.03672.

[77] pygridsynth developers, pygridsynth: a Python implementation of the Ross–Selinger algorithm, version 2.0.0. [TODO: cite the proper software reference/DOI].

[78] F. Vatan and C. Williams, Optimal quantum circuits for general two-qubit gates, Phys. Rev. A **69**, 032315 (2004), arXiv:quant-ph/0308006.

[79] M. A. Nielsen and I. L. Chuang, Quantum Computation and Quantum Information, 10th ed. (Cambridge University Press, Cambridge, 2010).

[80] G. Evenbly and G. Vidal, Algorithms for entanglement renormalization, Phys. Rev. B **79**, 144108 (2009), arXiv:0707.1454.

[81] K. M. Nakanishi, K. Fujii, and S. Todo, Sequential minimal optimization for quantum-classical hybrid algorithms, Phys. Rev. Research **2**, 043158 (2020), arXiv:1903.12166.

[82] M. Ostaszewski, E. Grant, and M. Benedetti, Structure optimization for parameterized quantum circuits, Quantum **5**, 391 (2021), arXiv:1905.09692.

[83] D. Wierichs, J. Izaac, C. Wang, and C. Y.-Y. Lin, General parameter-shift rules for quantum gradients, Quantum **6**, 677 (2022), arXiv:2107.12390.